Lokakarya

Coding Python

Array & Fungsi untuk Pemrograman Modular

Disusun Oleh
Narasumber UPT TIKP



Tujuan

- Memahami array dan aplikasinya
- Membuat, mengakses, dan memodifikasi array
- Menggunakan loop dengan array untuk penanganan data yang efisien
- Menulis dan memanfaatkan fungsi untuk pemrograman modular
- Melewati argumen dan mengembalikan nilai
- Membangun program interaktif menggunakan array dan fungsi

Mengapa Menggunakan Array?

- **Penyimpanan Data yang Efisien:** Simpan beberapa nilai di bawah satu nama variabel
- **Akses Mudah:** Ambil elemen menggunakan indeks
- **Performa Lebih Baik:** Memproses data dalam jumlah besar secara efisien
- **Loop yang Dioptimalkan:** Lakukan operasi pada beberapa elemen dengan loop

Array dalam Python

Larik (Array) adalah struktur data yang menyimpan banyak nilai dalam satu variabel. Tidak seperti variabel biasa yang hanya menyimpan satu nilai, larik memungkinkan kita menyimpan, mengakses, dan memanipulasi data dalam jumlah besar secara efisien. Larik berguna saat bekerja dengan daftar angka, nama, objek, atau data terstruktur lainnya .

Dalam Python, array dapat diimplementasikan menggunakan list (bawaan) atau modul array (untuk kasus khusus). Namun, list lebih umum digunakan karena lebih fleksibel.

Membuat Array (Menggunakan List di Python)

Python tidak memiliki tipe data array bawaan seperti beberapa bahasa pemrograman lainnya. Sebagai gantinya, kami menggunakan **list** , yang berfungsi seperti array dinamis.

Sintaksis:

```
my_array = [10, 20, 30, 40, 50]  # Creating an array (list)
```

Mengakses Elemen dalam Array

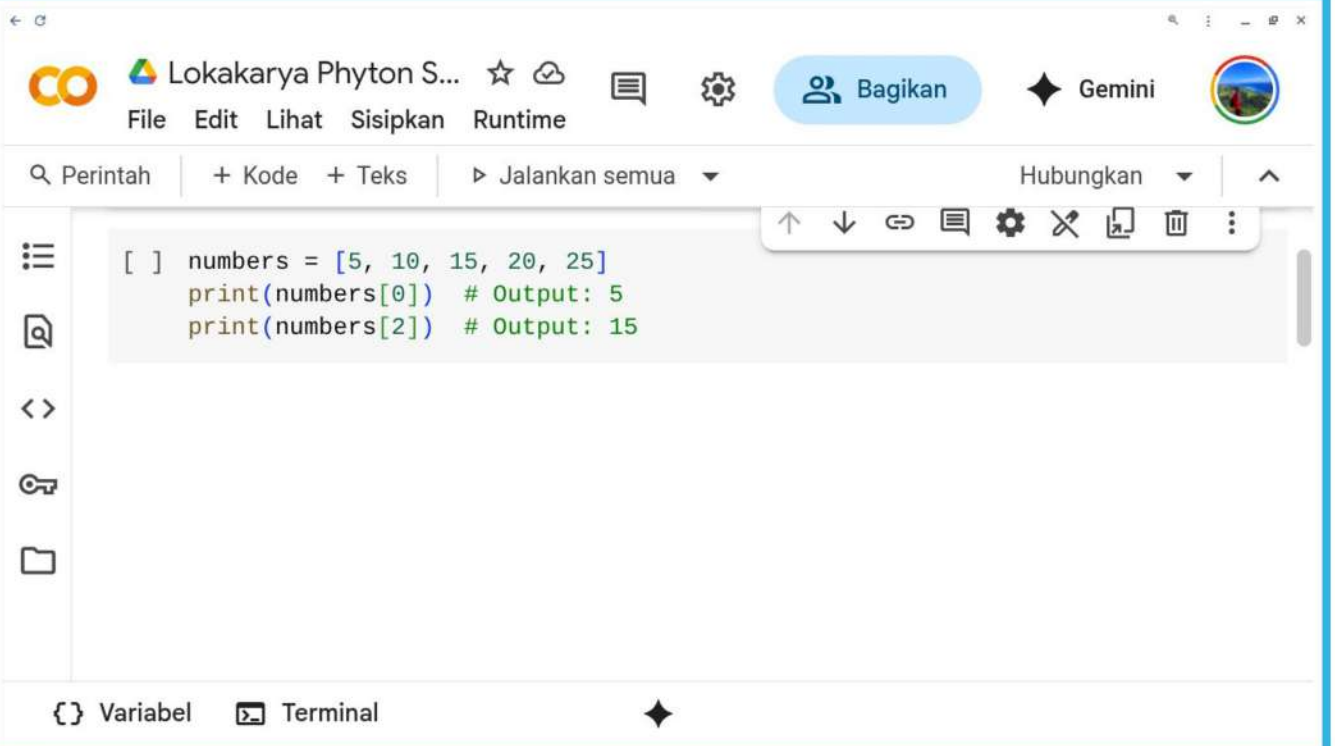
Kita dapat mengakses elemen array menggunakan **indeks** , di mana elemen pertama dimulai pada **indeks 0** .

Contoh:

```
numbers = [5, 10, 15, 20, 25]
print(numbers[0])   # Output: 5
print(numbers[2])   # Output: 15
```

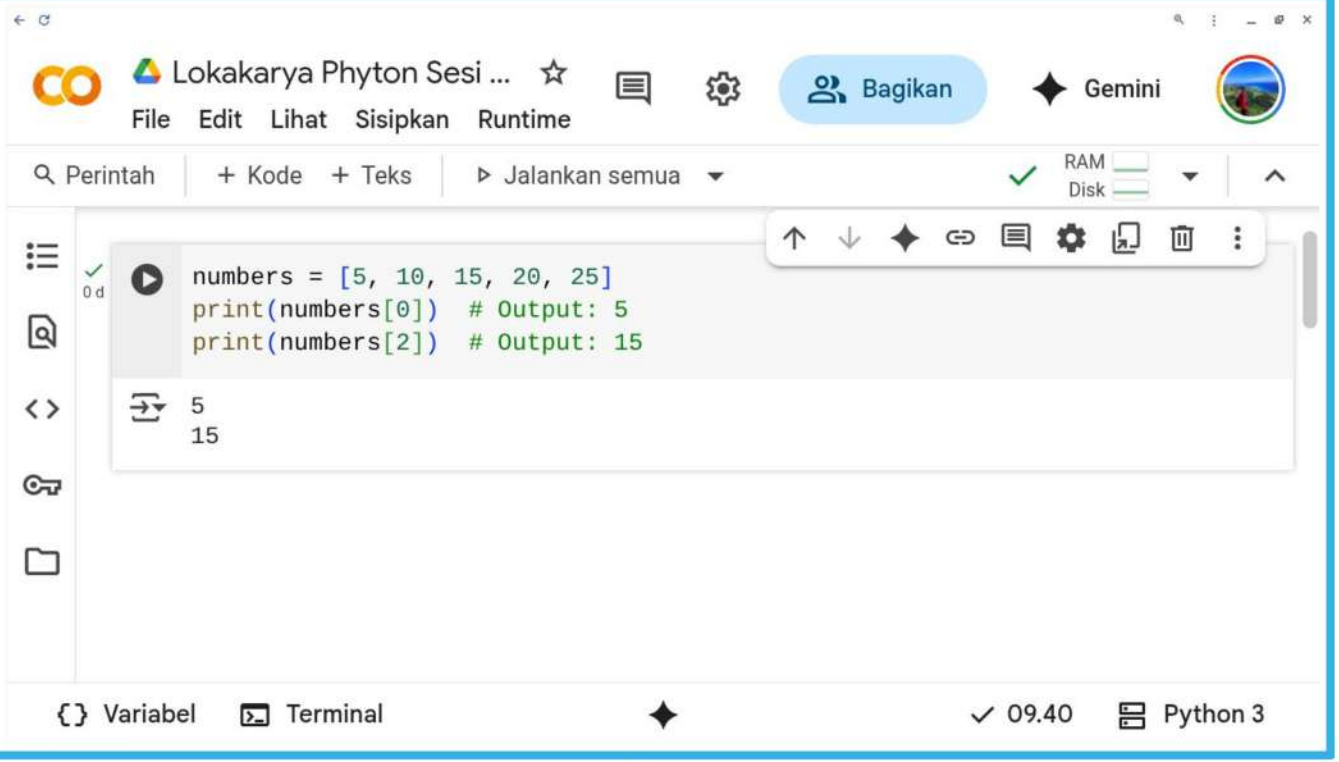
Mengakses Elemen dalam Array

Array Angka



Mengakses Elemen dalam Array

Hasil Cetak Array Angka



Memodifikasi Array

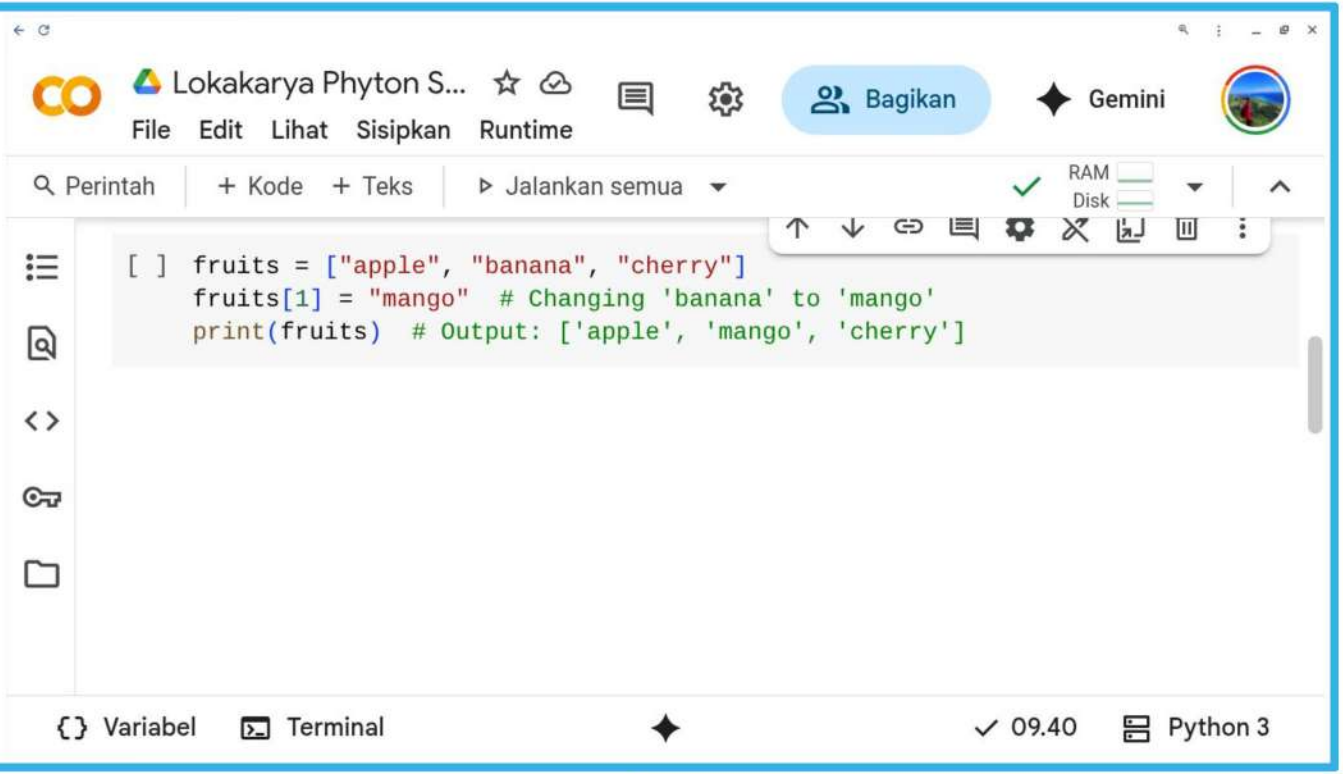
Anda dapat **memperbarui, menambah, atau menghapus elemen** dalam suatu array.

Contoh:

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "mango"  # Changing 'banana' to 'mango'
print(fruits)  # Output: ['apple', 'mango', 'cherry']
```

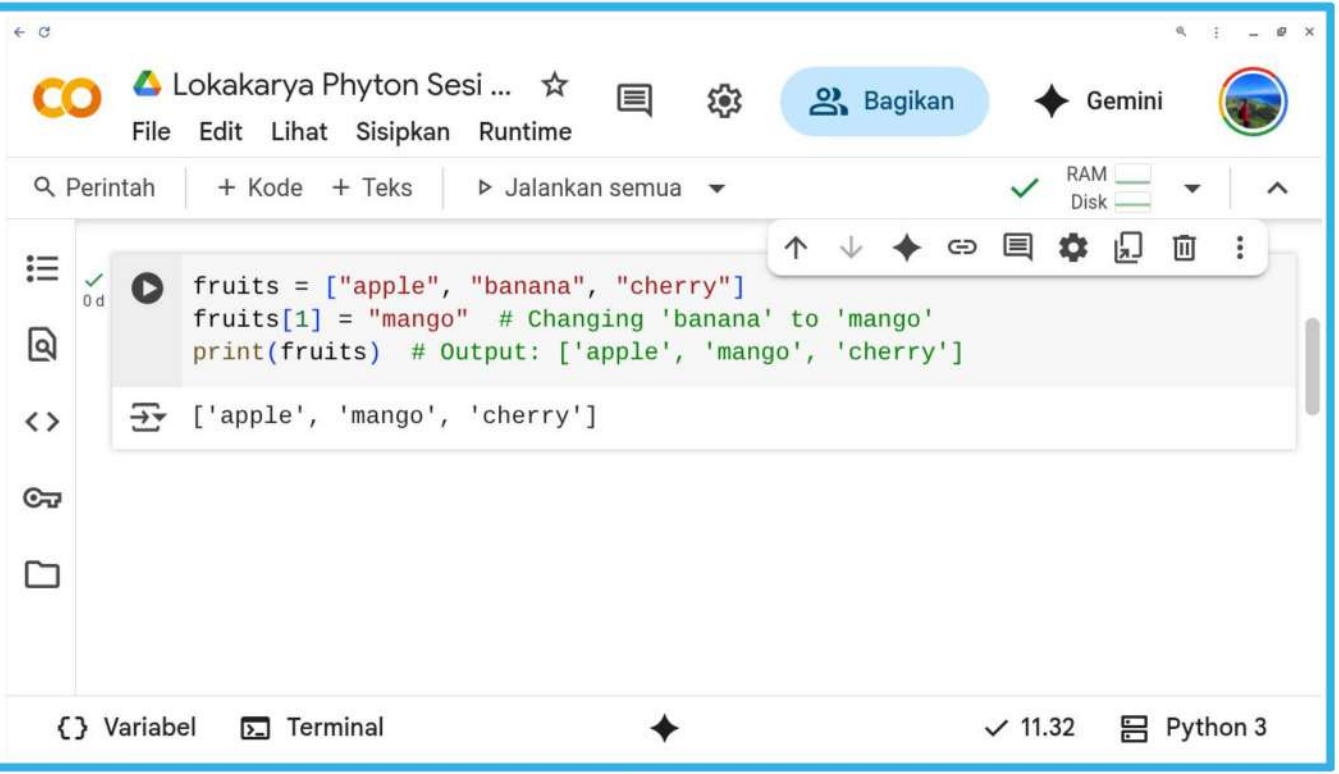
Memodifikasi Array

Mengubah Elemen Array



Memodifikasi Array

Nama Elemen Pada Array Berubah



Perulangan Melalui Array

Perulangan memudahkan pengerjaan kumpulan data besar.

Contoh (menggunakan for loop):

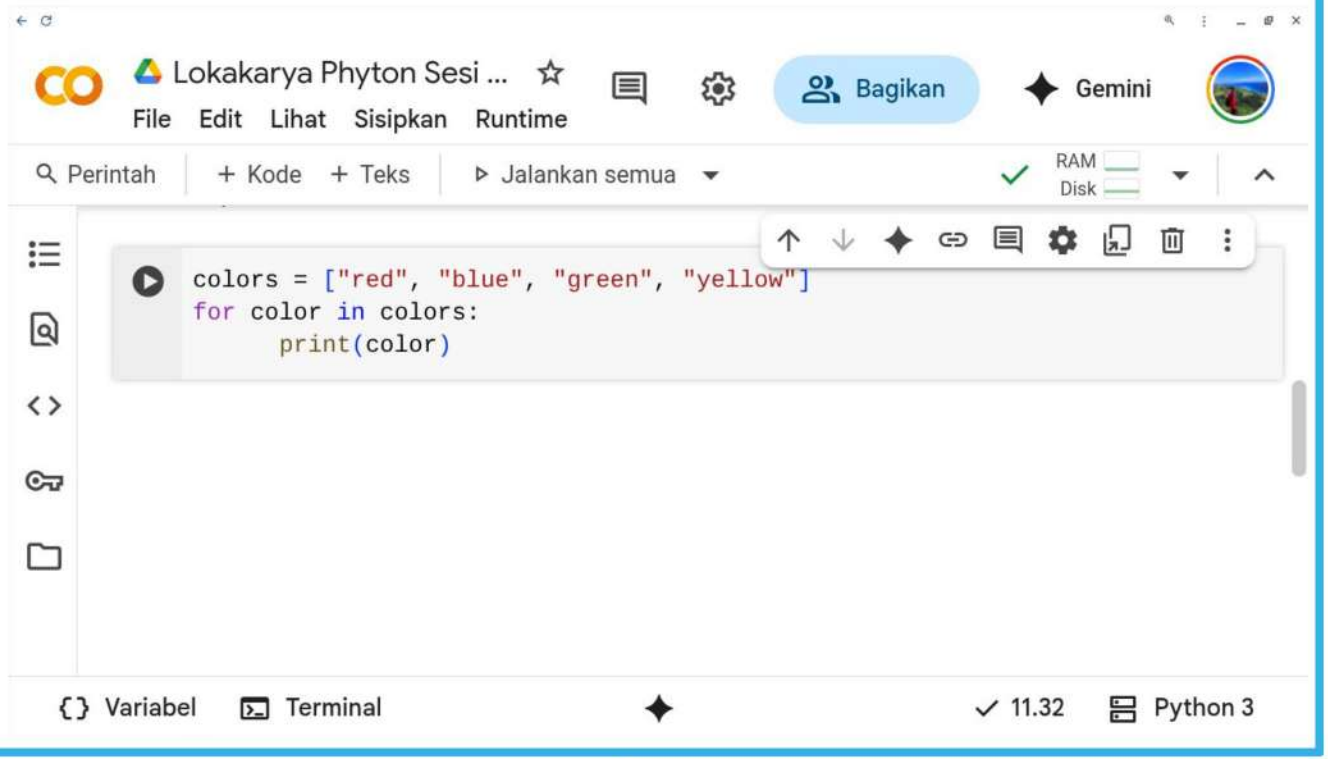
```
colors = ["red", "blue", "green", "yellow"]

for color in colors:

    print(color)
```

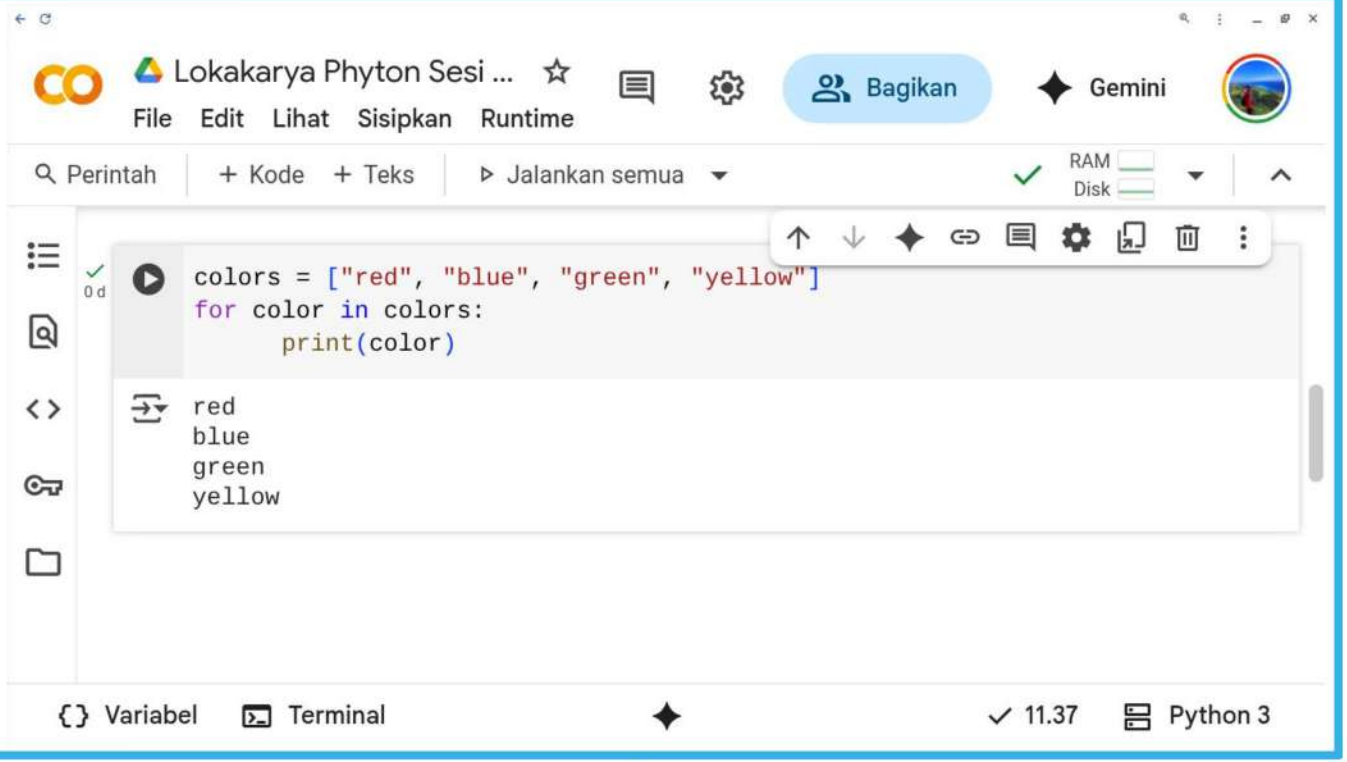
Perulangan Melalui Array

Perulangan Menggunakan For



Perulangan Melalui Array

Hasil Perulangan Menggunakan For



Perulangan Melalui Array

Perulangan memudahkan pengerjaan kumpulan data besar.

Contoh (menggunakan while loop):

```
numbers = [10, 20, 30, 40]

i = 0

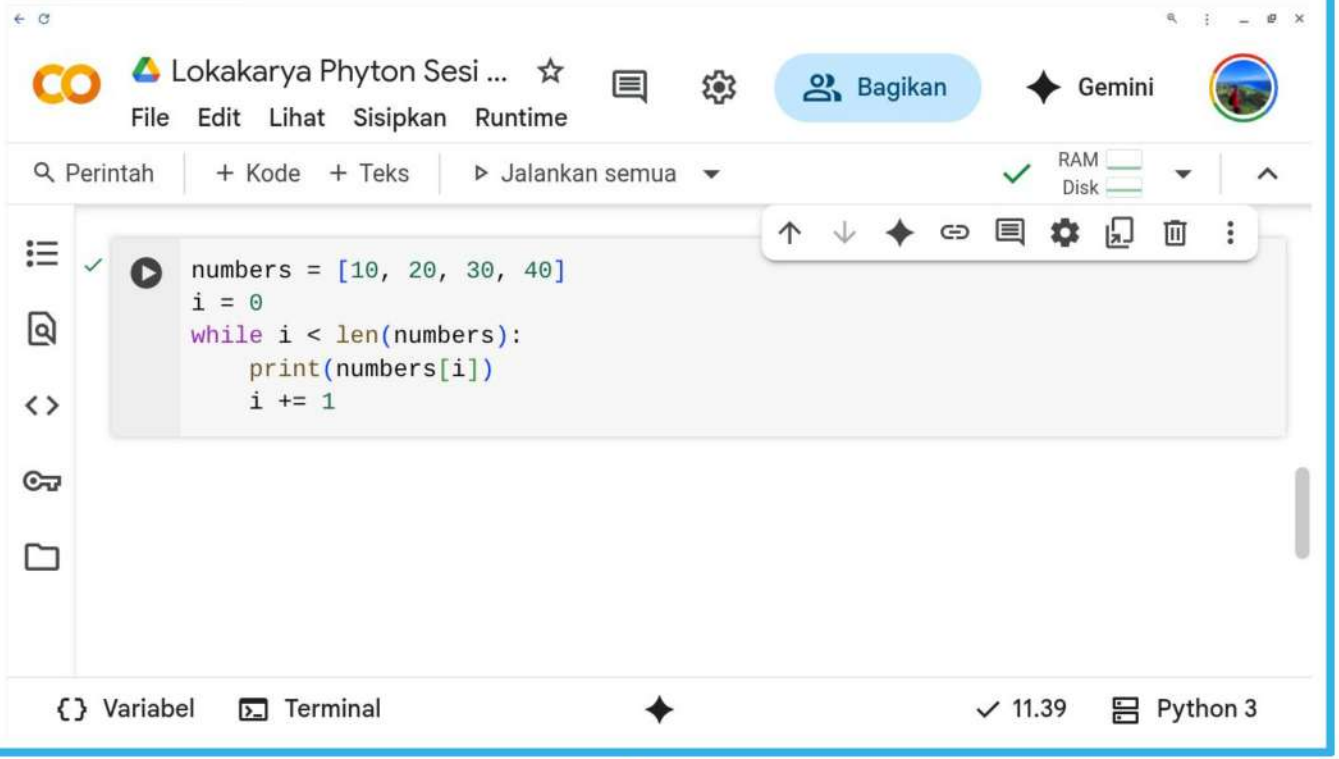
while i < len(numbers):

    print(numbers[i])

        i += 1
```

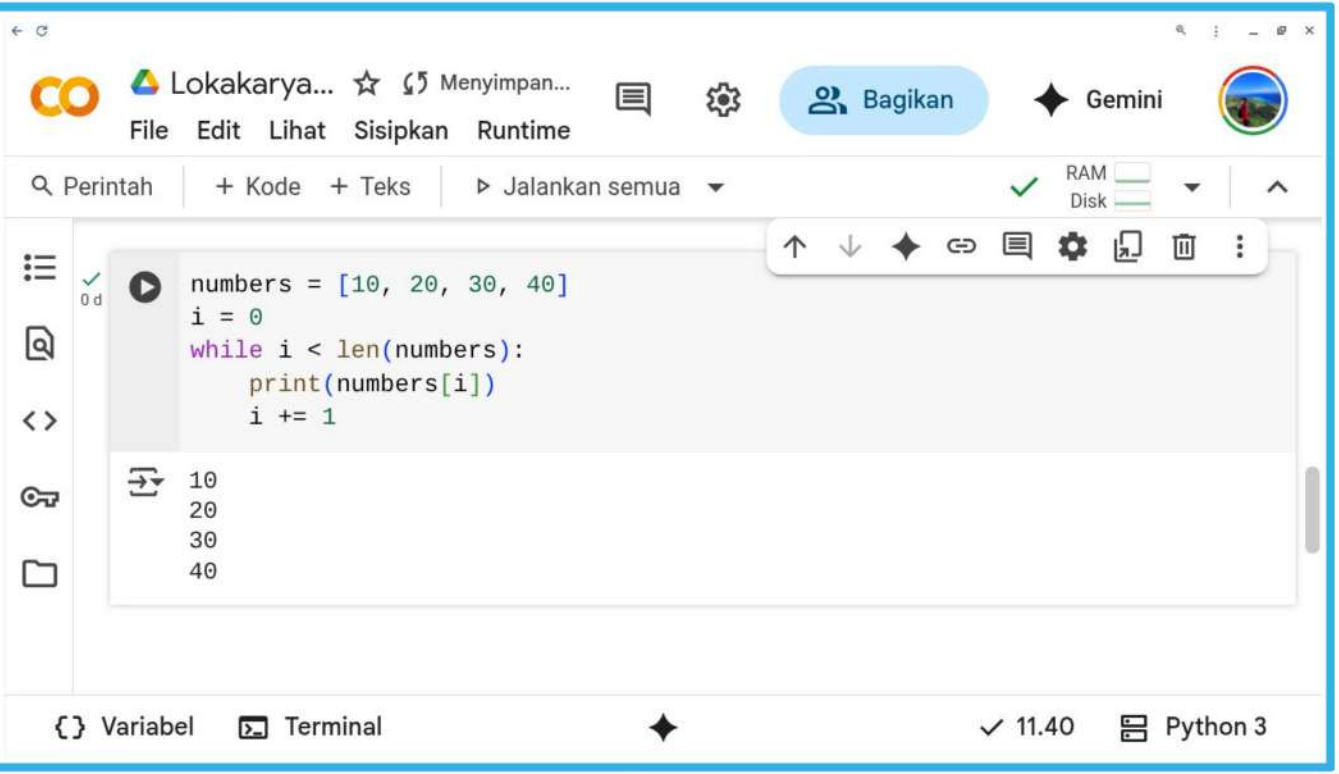
Perulangan Melalui Array

Perulangan Menggunakan While



Perulangan Melalui Array

Hasil Perulangan Menggunakan While



Contoh Umum: Mengelola Nilai Siswa

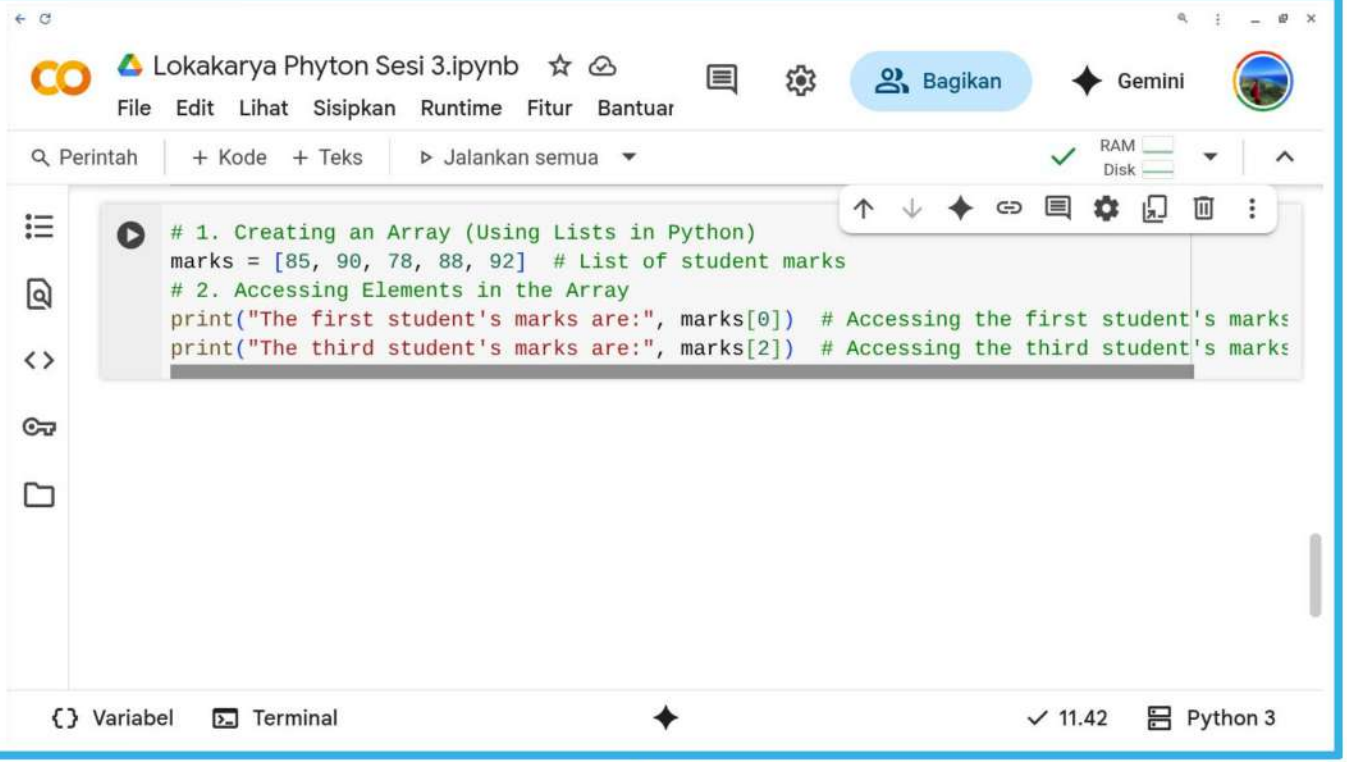
Mari kita buat program sederhana untuk mengelola dan memanipulasi daftar nilai siswa. Contoh ini akan mencakup **pembuatan array** , **mengakses elemen** , **memodifikasi array** , dan **melakukan perulangan pada array** .

Contoh Umum: Mengelola Nilai Siswa

```
# 1. Creating an Array (Using Lists in Python)
marks = [85, 90, 78, 88, 92]  # List of student marks
# 2. Accessing Elements in the Array
print("The first student's marks are:", marks[0])  # Accessing the first student's marks
print("The third student's marks are:", marks[2])  # Accessing the third student's marks
```

Contoh Umum: Mengelola Nilai Siswa

Penggunaan Array pada pengelolaan Nilai

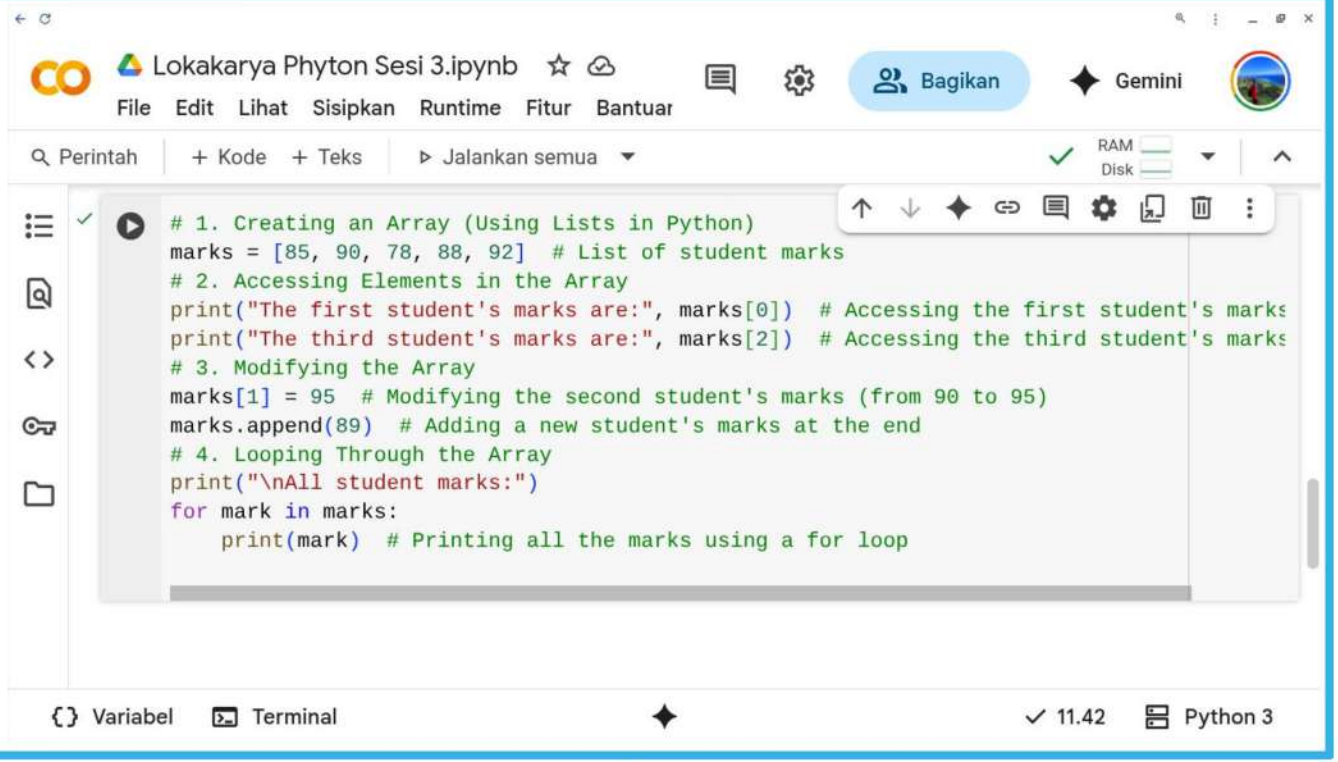


Contoh Umum: Mengelola Nilai Siswa

```
# 3. Modifying the Array
marks[1] = 95  # Modifying the second student's marks (from
90 to 95)
marks.append(89)  # Adding a new student's marks at the end
# 4. Looping Through the Array
print("\nAll student marks:")
for mark in marks:
    print(mark)  # Printing all the marks using a for loop
```

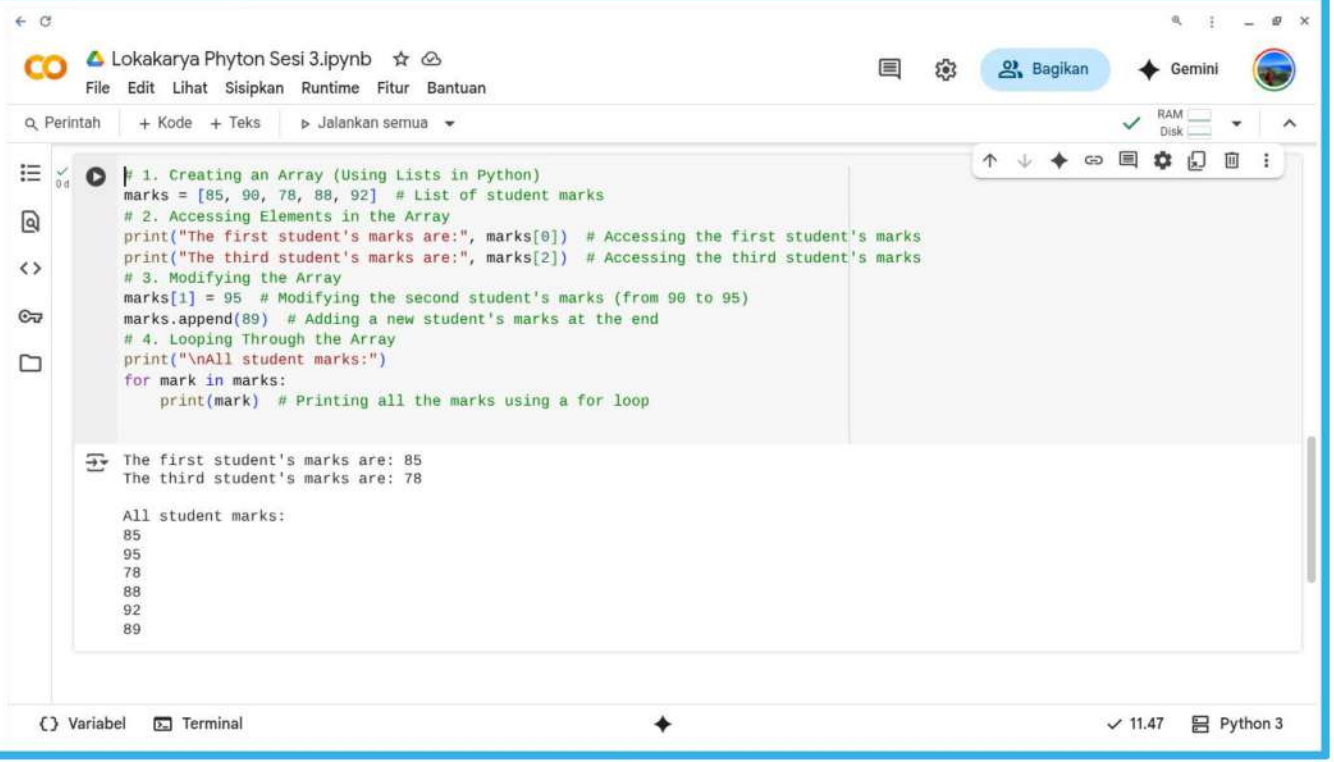
Contoh Umum: Mengelola Nilai Siswa

Penggunaan Array pada pengelolaan Nilai



Contoh Umum: Mengelola Nilai Siswa

Hasil Penggunaan Array pada pengelolaan Nilai



Aplikasi Umum Array

1. **Penyimpanan Data:** Menyimpan daftar nama, skor, atau item inventaris
2. **Pengembangan Game:** Mengelola statistik pemain, posisi musuh, atau level
3. **Komputasi Ilmiah:** Menangani kumpulan data besar dalam analisis numerik
4. **Pembelajaran Mesin:** Mengelola kumpulan data untuk pelatihan model
5. **Sistem Keuangan:** Menyimpan harga saham dan catatan transaksi

Menulis dan Memanfaatkan Fungsi

Dalam Python, **fungsi** memungkinkan Anda mengelompokkan serangkaian instruksi yang menjalankan tugas tertentu. Hal ini memungkinkan Anda untuk memanggil instruksi tersebut kapanpun dibutuhkan, sehingga meningkatkan **penggunaan ulang kode** dan mengurangi redudansi dalam program Anda. Fungsi adalah konsep dasar dalam Python yang membantu mengorganisasikan kode secara efisien, sehingga lebih mudah dibaca, dikelola, dan diuji.

Apa itu Fungsi?

Fungsi adalah blok kode yang dapat digunakan kembali yang hanya berjalan ketika dipanggil. Anda dapat meneruskan data ke fungsi melalui parameter dan menerima hasil menggunakan nilai balik .

Sintaks Fungsi

```
def function_name(parameter1, parameter2):
    # Code block
    result = parameter1 + parameter2  # Example of logic
    return result
```

- `def`: Kata kunci yang digunakan untuk mendefinisikan suatu fungsi.
- `function_name`: Nama fungsi. Nama tersebut harus menggambarkan fungsinya.
- **Parameter** : Nilai yang diteruskan ke fungsi yang dapat digunakan untuk melakukan tugas.
- `return`: Mengirim kembali hasil fungsi ke pemanggil.

Membuat Fungsi

Mari kita mulai dengan membuat fungsi sederhana untuk menjumlahkan dua angka.

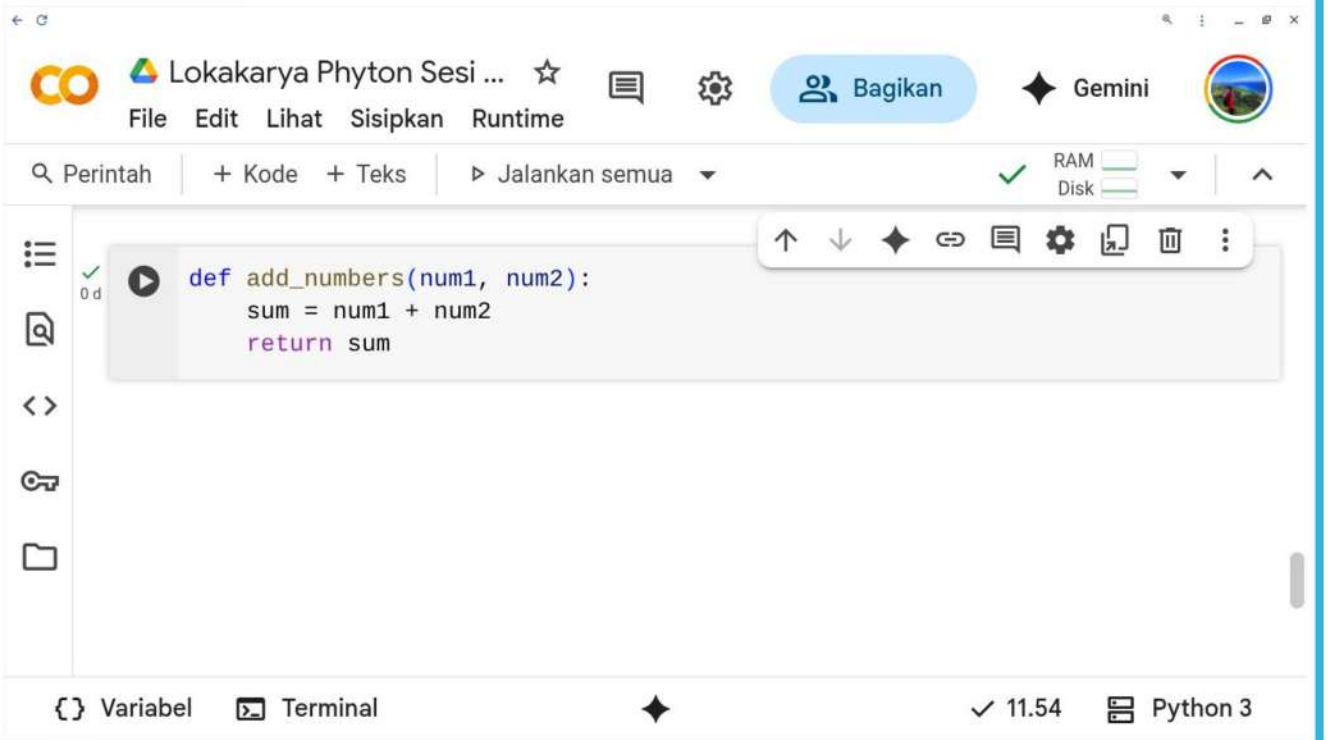
```
def add_numbers(num1, num2):
    sum = num1 + num2
    return sum
```

Di Sini:

- Fungsi tersebut `add_numbers` mengambil dua parameter, `num1` dan `num2`.
- Ia melakukan penjumlahan dan mengembalikan hasilnya.

Membuat Fungsi

Membuat fungsi Penjumlahan



Memanggil Fungsi

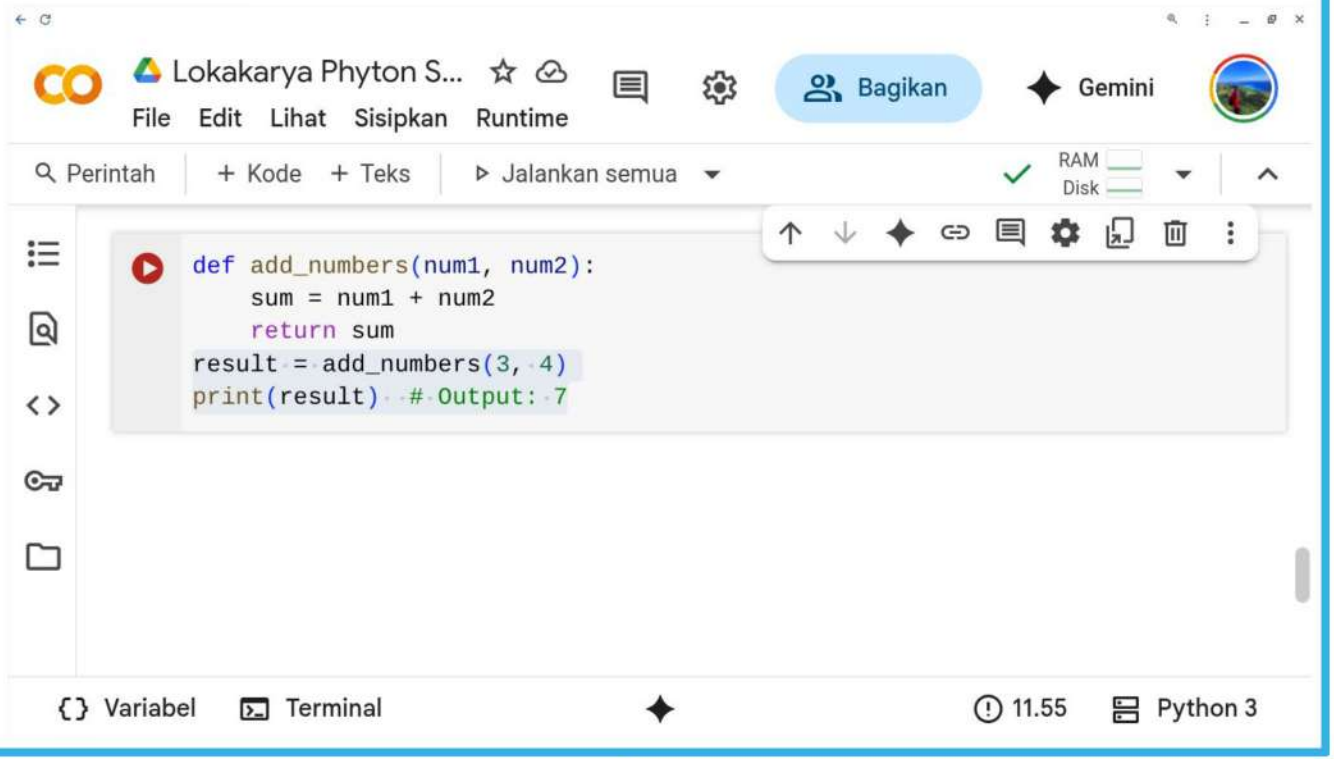
Untuk menggunakan fungsi yang baru kita buat, kita perlu memanggilnya:

```
result = add_numbers(3, 4)
print(result)  # Output: 7
```

- Fungsi tersebut `add_numbers(3, 4)` dipanggil dengan argumen `3` dan `4`.
- Nilai yang dikembalikan `7` disimpan dalam variabel `result` dan dicetak.

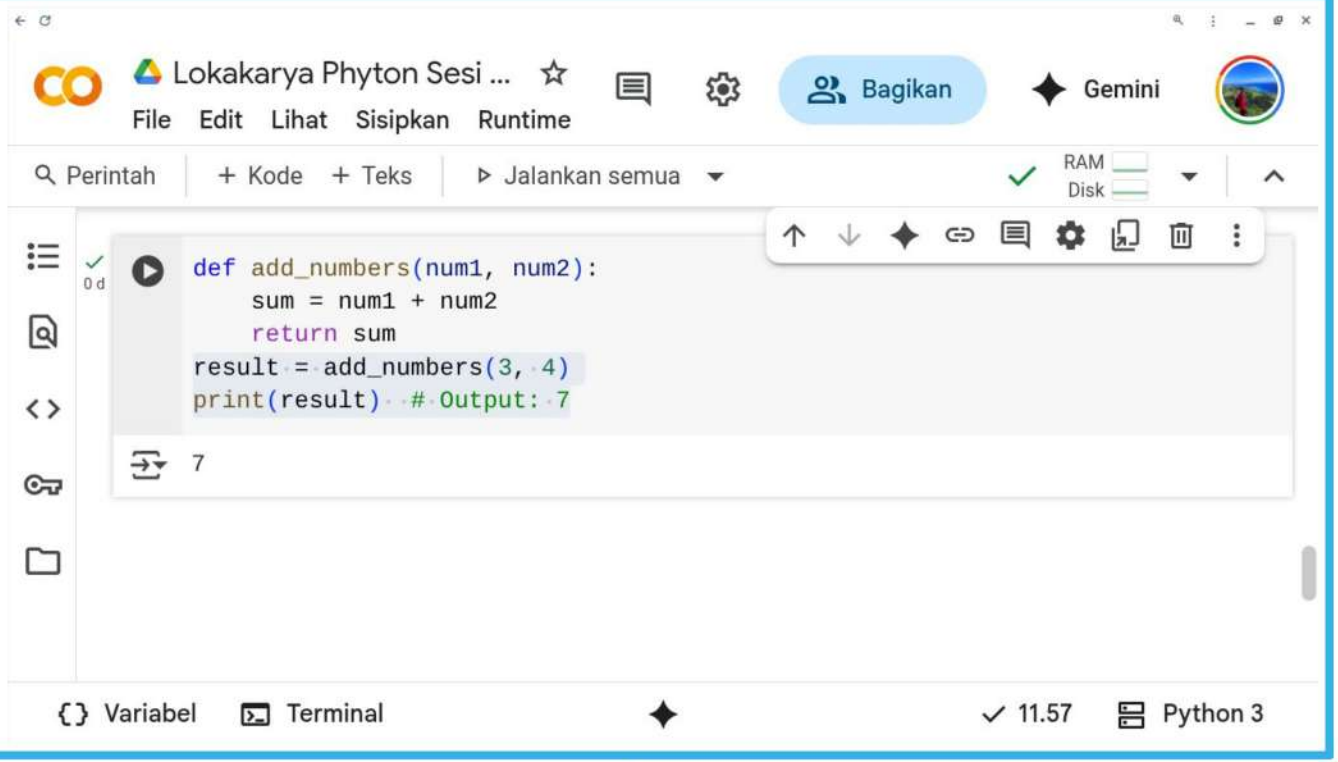
Memanggil Fungsi

Memanggil Fungsi untuk digunakan



Memanggil Fungsi

Hasil Memanggil Fungsi untuk digunakan



Menggunakan Fungsi untuk Kegunaan Ulang

Sekarang setelah kita tahu cara membuat dan menggunakan suatu fungsi, kita dapat memanggilnya beberapa kali tanpa perlu menulis logika penjumlahan setiap saat.

Contoh:

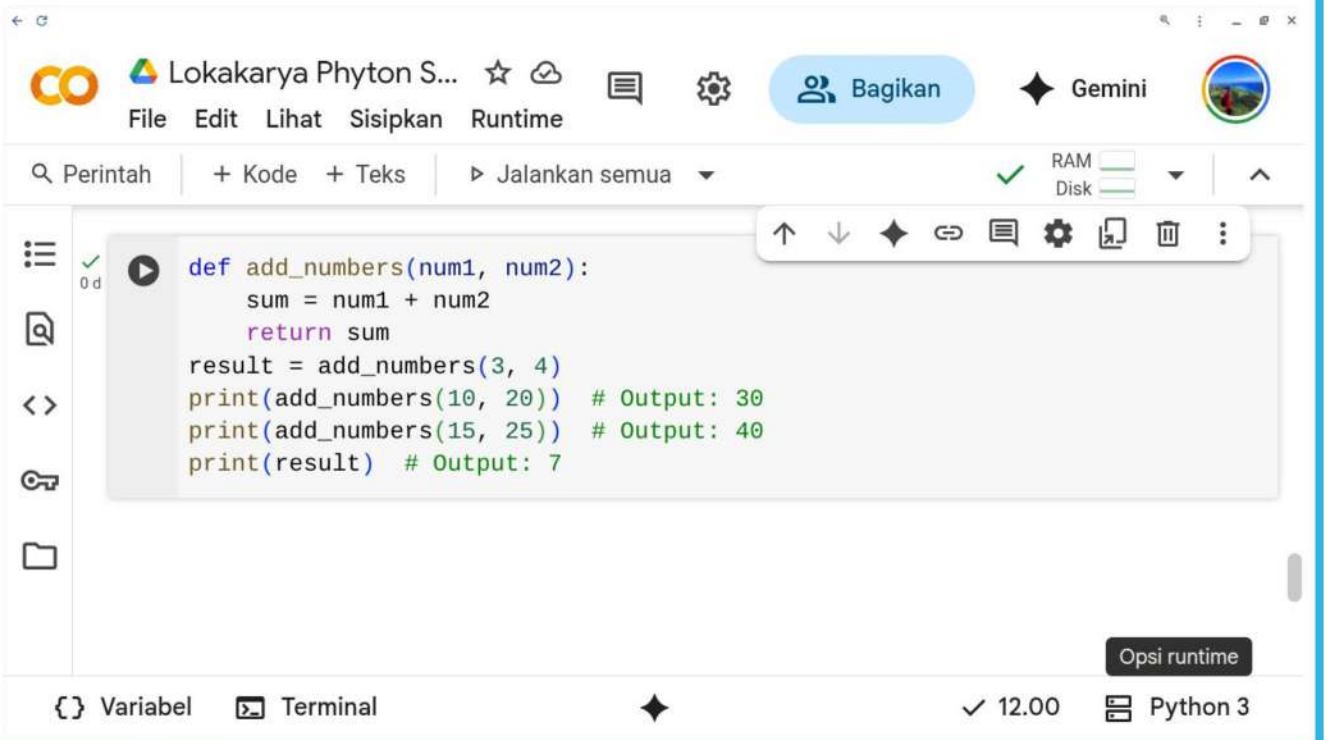
```
print(add_numbers(10, 20))   # Output: 30
print(add_numbers(15, 25))   # Output: 40
```

Dalam contoh ini:

- Kami menggunakan kembali `add_numbers` fungsi yang sama untuk melakukan penambahan dengan nilai yang berbeda.
- Pendekatan ini meminimalkan kode yang berulang.

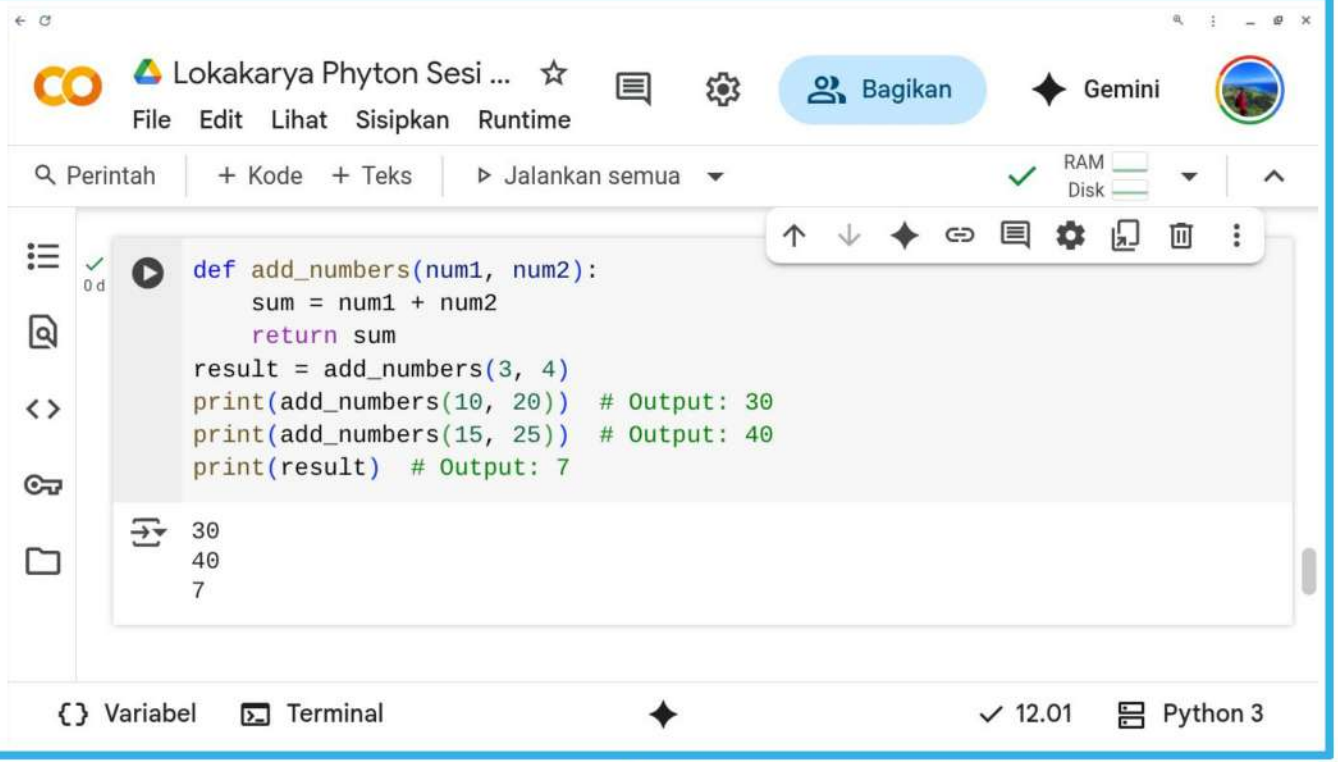
Menggunakan Fungsi untuk Kegunaan Ulang

Penggunaan Fungsi Lebih Dari Satu Kali



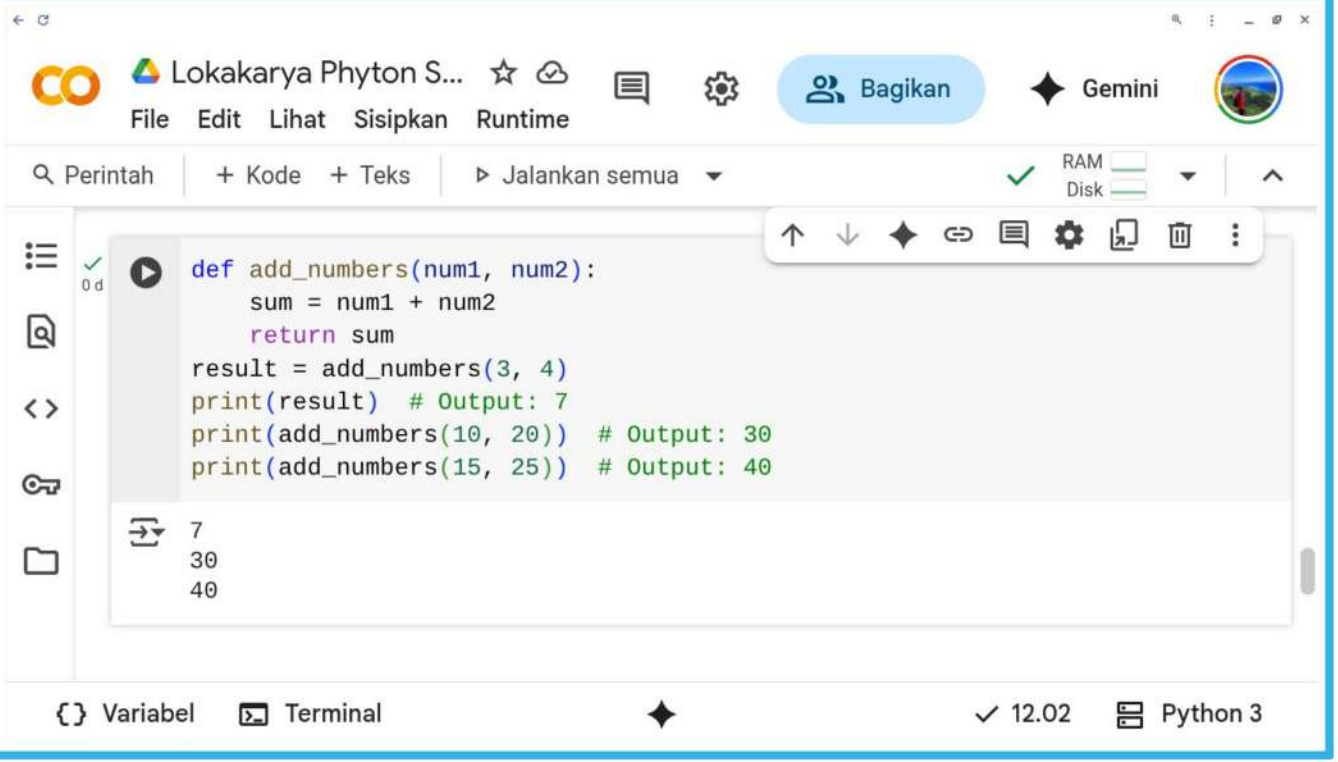
Menggunakan Fungsi untuk Kegunaan Ulang

Hasil Penggunaan Fungsi Lebih Dari Satu Kali



Menggunakan Fungsi untuk Kegunaan Ulang

Hasil Penggunaan Fungsi Lebih Dari Satu Kali



Fungsi dengan Beberapa Parameter

Fungsi dapat memiliki lebih dari satu parameter. Misalnya, mari kita buat fungsi yang menghitung luas persegi panjang:

```
def calculate_area(length, width):
    area = length * width
    return area
```

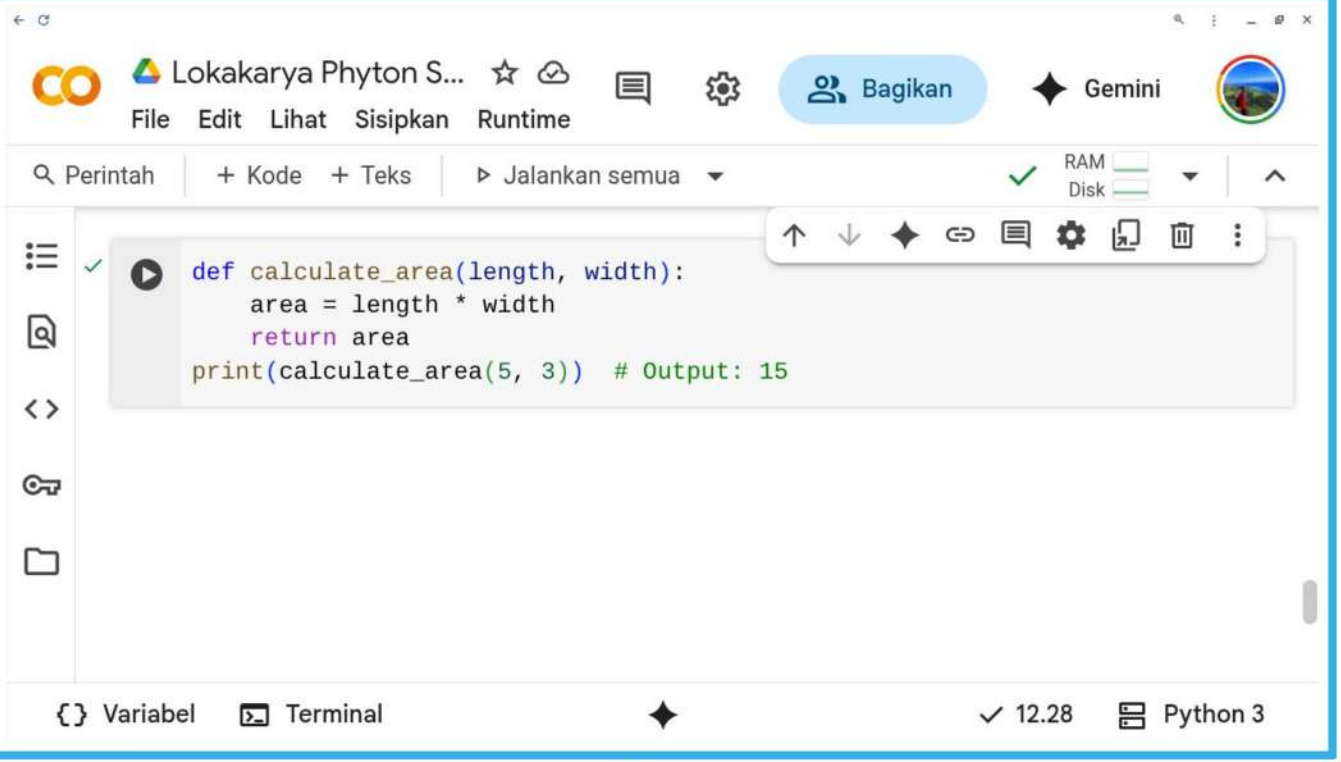
Untuk memanggil fungsi:

```
print(calculate_area(5, 3))  # Output: 15
```

Di sini, kita memberikan dua nilai, `length` dan `width`, dan fungsi tersebut mengembalikan hasil perkaliannya, yaitu luas persegi panjang.

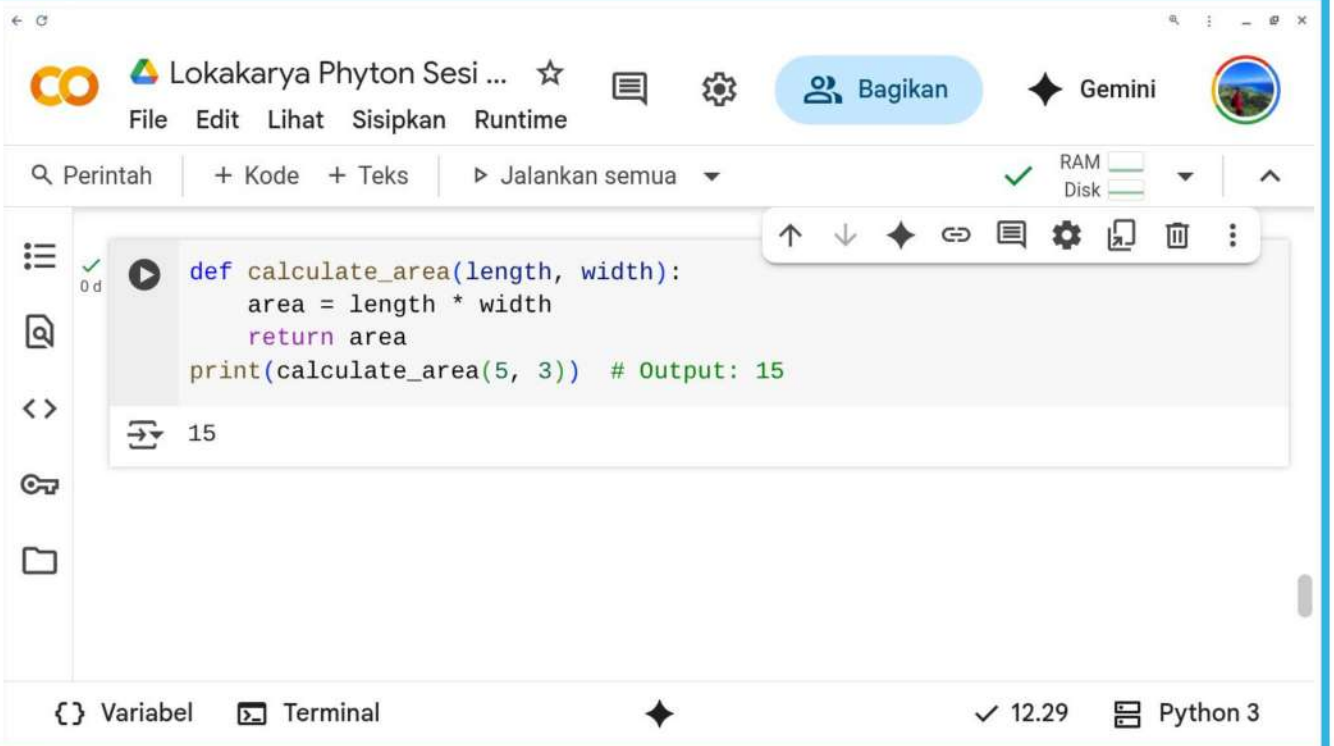
Fungsi dengan Beberapa Parameter

Menghitung luas persegi panjang



Fungsi dengan Beberapa Parameter

Hasil Menghitung luas persegi panjang



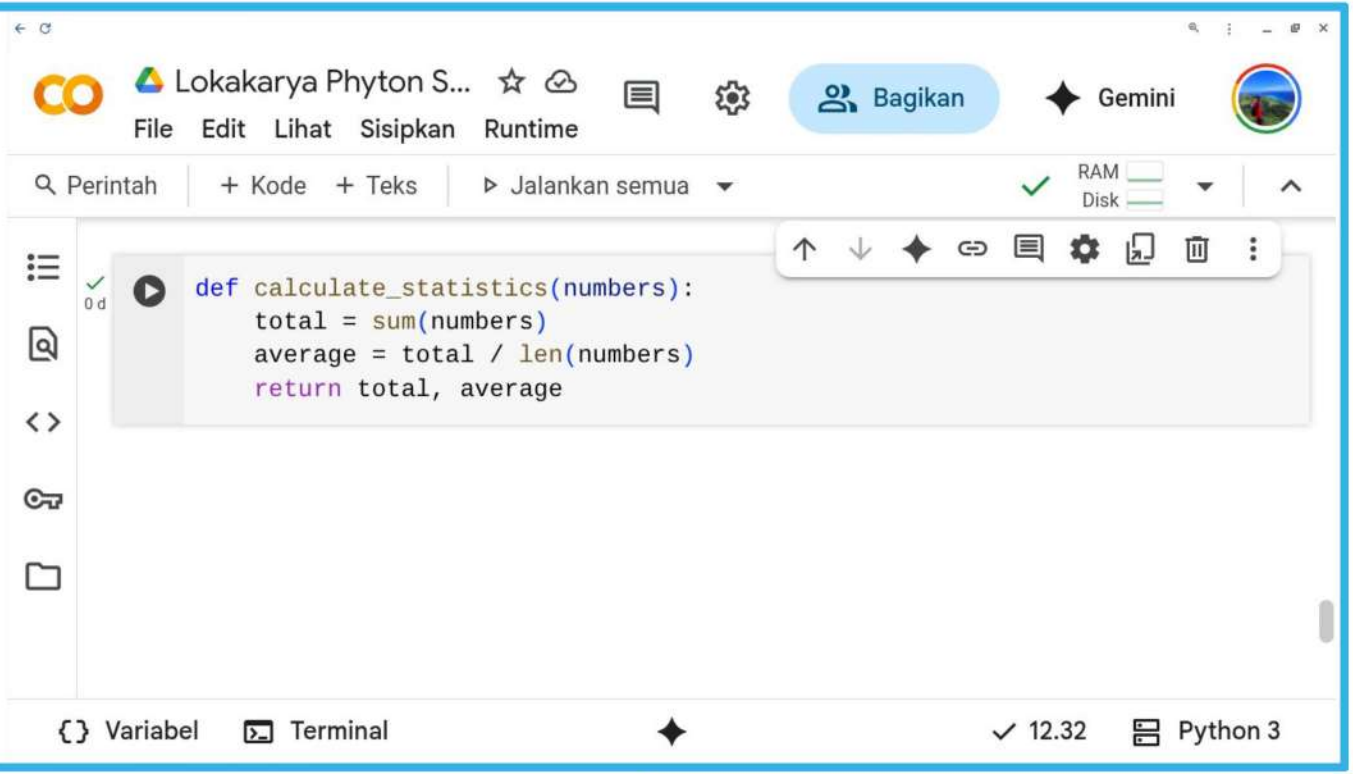
Mengembalikan Beberapa Nilai

Fungsi dapat mengembalikan beberapa nilai dengan menggunakan daftar.

```
def calculate_statistics(numbers):
    total = sum(numbers)
    average = total / len(numbers)
    return total, average
```

Mengembalikan Beberapa Nilai

Menuliskan fungsi



Mengembalikan Beberapa Nilai

Untuk memanggil fungsi:

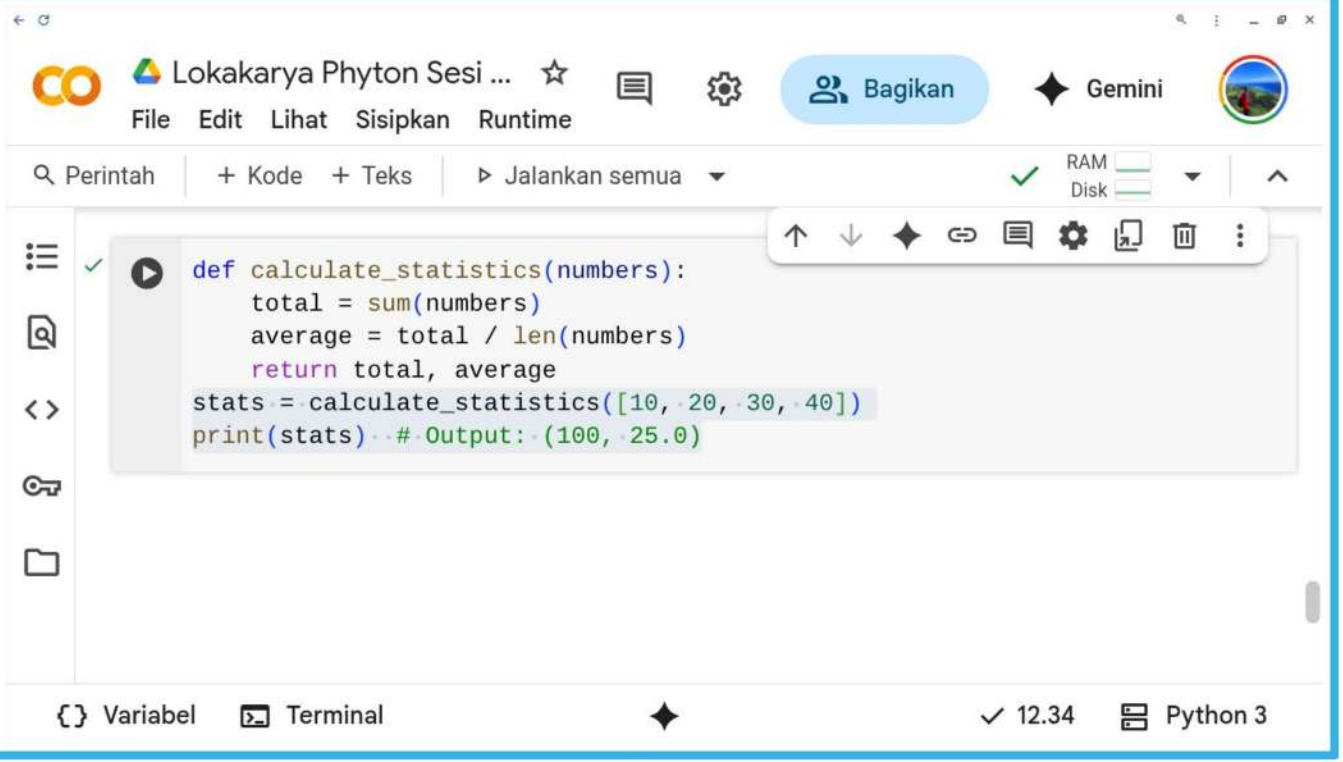
```
stats = calculate_statistics([10, 20, 30, 40])

print(stats)  # Output: (100, 25.0)
```

Di sini, fungsi mengembalikan daftar yang berisi total dan rata-rata, dan kita dapat menggunakan kedua nilai dari daftar yang dikembalikan.

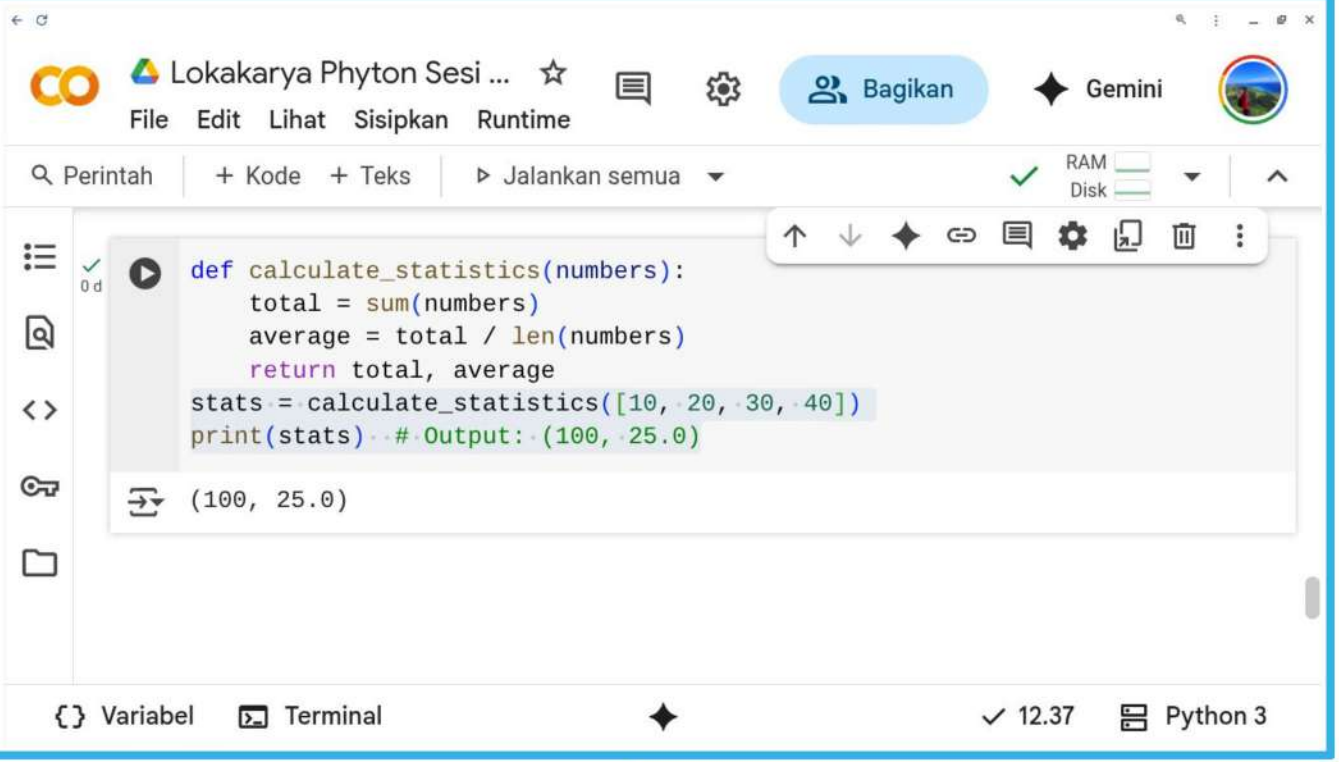
Mengembalikan Beberapa Nilai

Menggunakan Fungsi Untuk Melakukan Pengolahan Angka



Mengembalikan Beberapa Nilai

Hasil Menggunakan Fungsi Untuk Melakukan Pengolahan Angka



Manfaat Menggunakan Fungsi

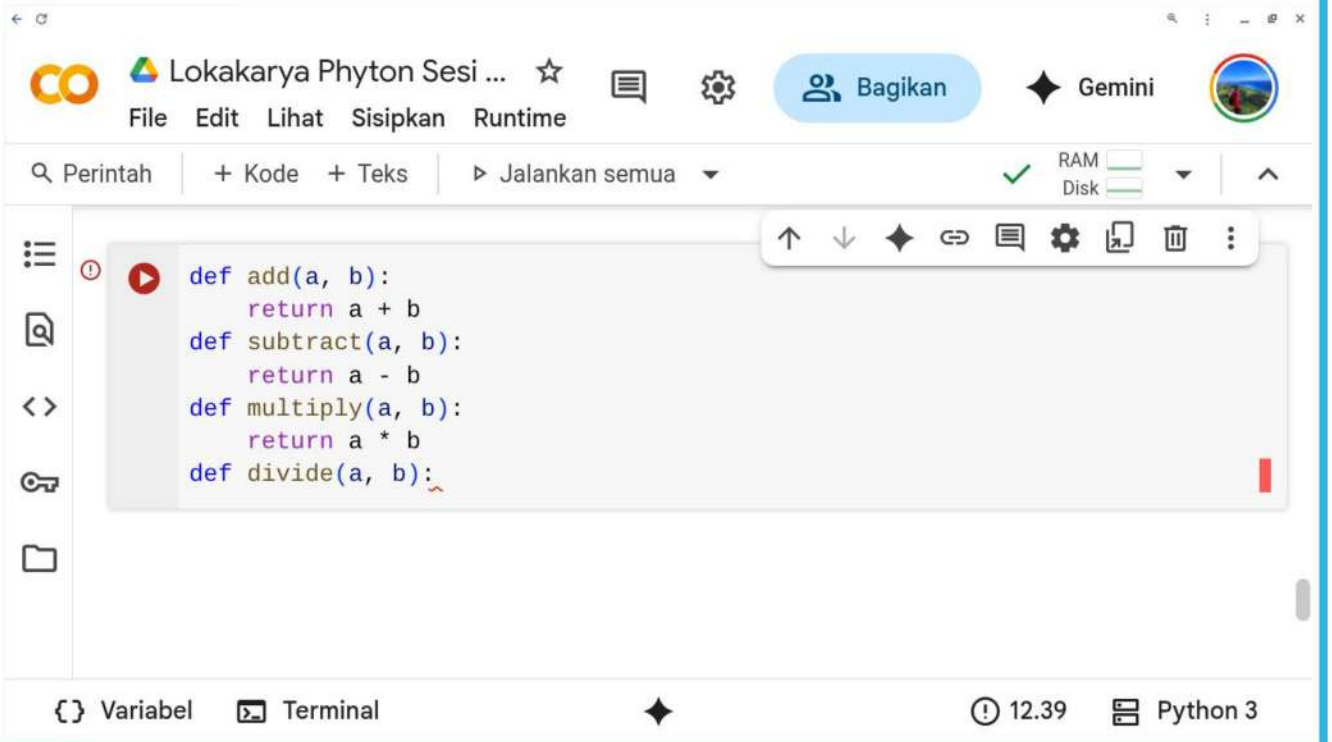
1. **Dapat Digunakan Kembali Kode** : Fungsi dapat digunakan beberapa kali, mengurangi kebutuhan untuk menulis kode yang sama lagi.
2. **Organisasi** : Memecah masalah yang rumit menjadi fungsi yang lebih kecil membuat kode lebih mudah dibaca.
3. **Pemeliharaan** : Memperbaiki bug atau memperbarui kode lebih mudah ketika logika dibagi menjadi fungsi.
4. **Modularitas** : Fungsi memungkinkan Anda membuat potongan logika yang mandiri, membuatnya lebih mudah untuk diuji dan di-debug.

Aktivitas: Kalkulator Berbasis Fungsi

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
```

Aktivitas: Kalkulator Berbasis Fungsi

Membuat Beberapa Fungsi Operasi Matematika

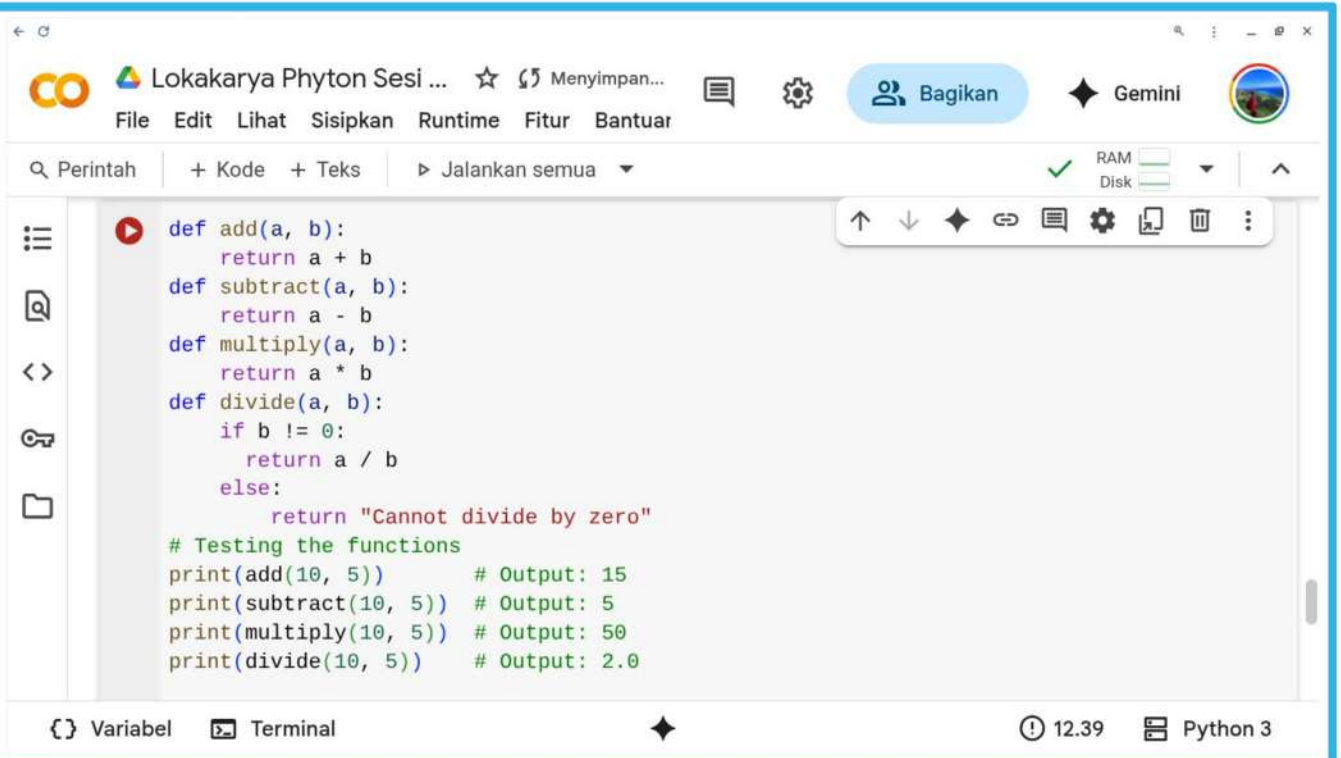


Aktivitas: Kalkulator Berbasis Fungsi

```
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"
# Testing the functions
print(add(10, 5))       # Output: 15
print(subtract(10, 5))  # Output: 5
print(multiply(10, 5))  # Output: 50
print(divide(10, 5))    # Output: 2.0
```

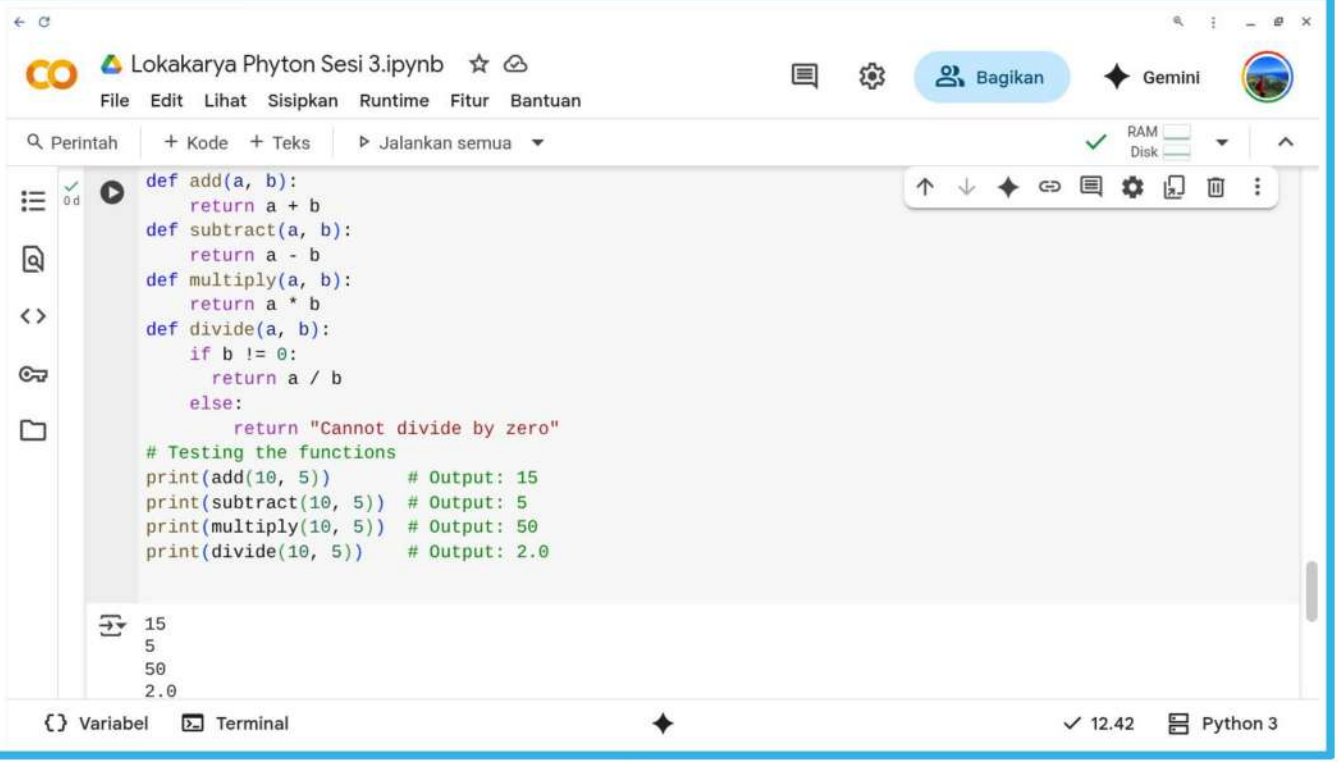
Aktivitas: Kalkulator Berbasis Fungsi

Membuat Kondisi Dilakukan Perhitungan



Aktivitas: Kalkulator Berbasis Fungsi

Cetak Hasil Perhitungan Sesuai Kondisi yang dipenuhi



Aktivitas: Array dalam Python

Tujuan:

- Temukan angka terbesar dalam suatu array.
- Hitung jumlah semua elemen dalam suatu array.
- Membalikkan susunannya.
- Periksa apakah nomor tertentu ada dalam array.

Aktivitas: Array dalam Python

Konsep yang Dicakup:

- Array (Daftar dalam Python)
- Mengulangi melalui array
- Fungsi dan metode bawaan dalam Python
- Pernyataan bersyarat

Aktivitas: Array dalam Python

Langkah 1: Buat Array

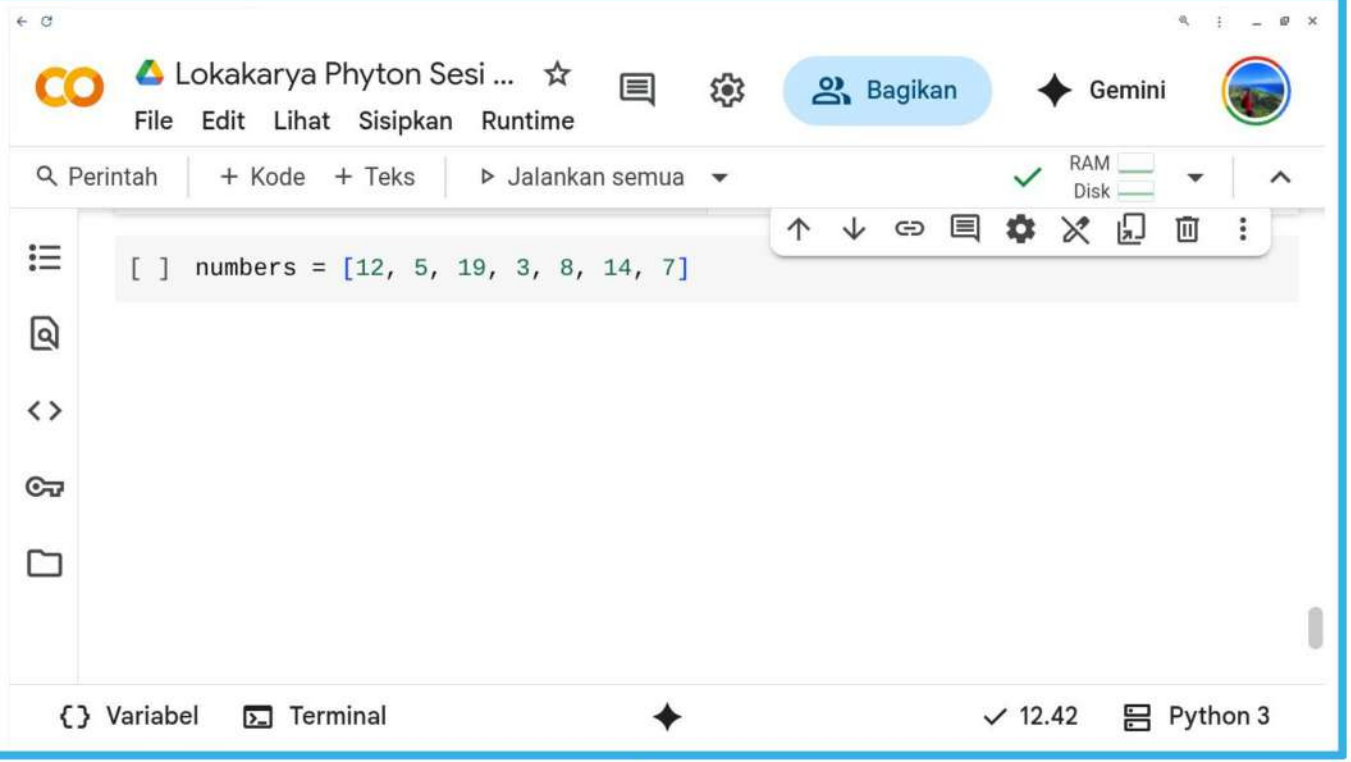
Mari kita mulai dengan membuat sebuah array (daftar) berisi beberapa angka. Kita akan melakukan berbagai operasi pada array ini.

```
numbers = [12, 5, 19, 3, 8, 14, 7]
```

Susunan ini berisi beberapa angka acak.

Aktivitas: Array dalam Python

Membuat Array



Aktivitas: Array dalam Python

Langkah 2: Menemukan Angka Terbesar dalam Array

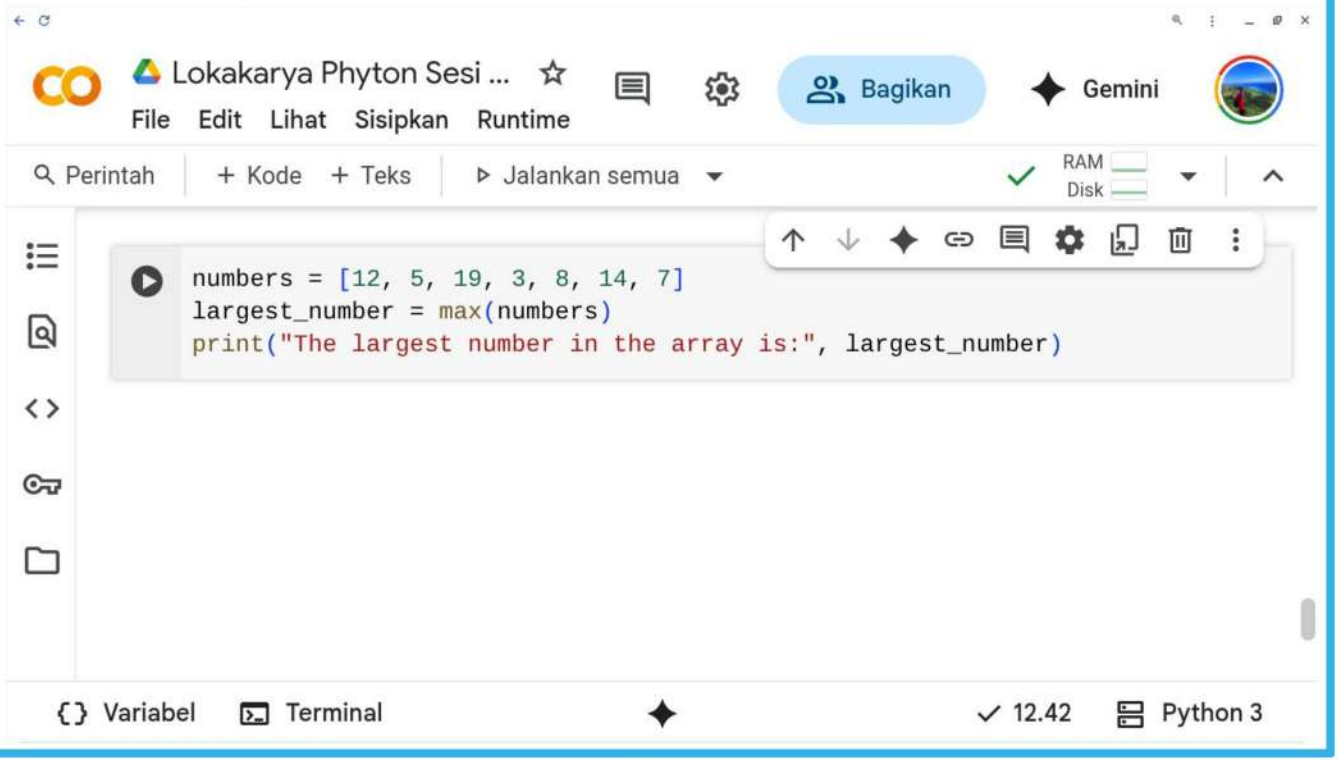
Untuk menemukan angka terbesar, kita dapat menggunakan `max()` fungsi. `max()` Fungsi ini mengembalikan nilai tertinggi dalam array.

```
largest_number = max(numbers)
print("The largest number in the array is:",
largest_number)
```

- **Penjelasan** : `max()` Fungsi ini memindai array dan mengembalikan nilai tertinggi.

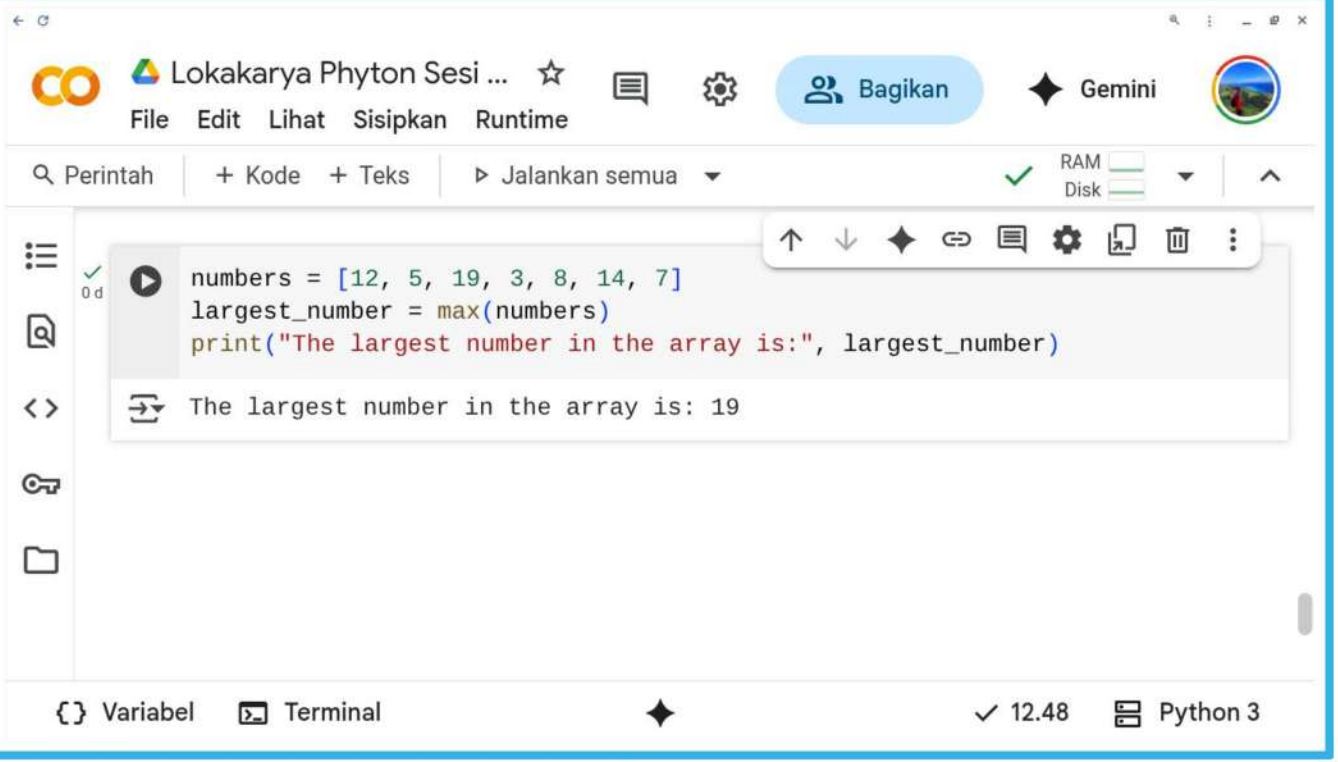
Aktivitas: Array dalam Python

Membuat Fungsi Angka Terbesar dan Mencetaknya



Aktivitas: Array dalam Python

Hasil Fungsi Angka Terbesar dan Mencetaknya



Aktivitas: Array dalam Python

Langkah 3: Menghitung Jumlah Array

Untuk menemukan jumlah semua elemen dalam array, kita akan menggunakan `sum()` fungsi. Fungsi ini menjumlahkan semua elemen.

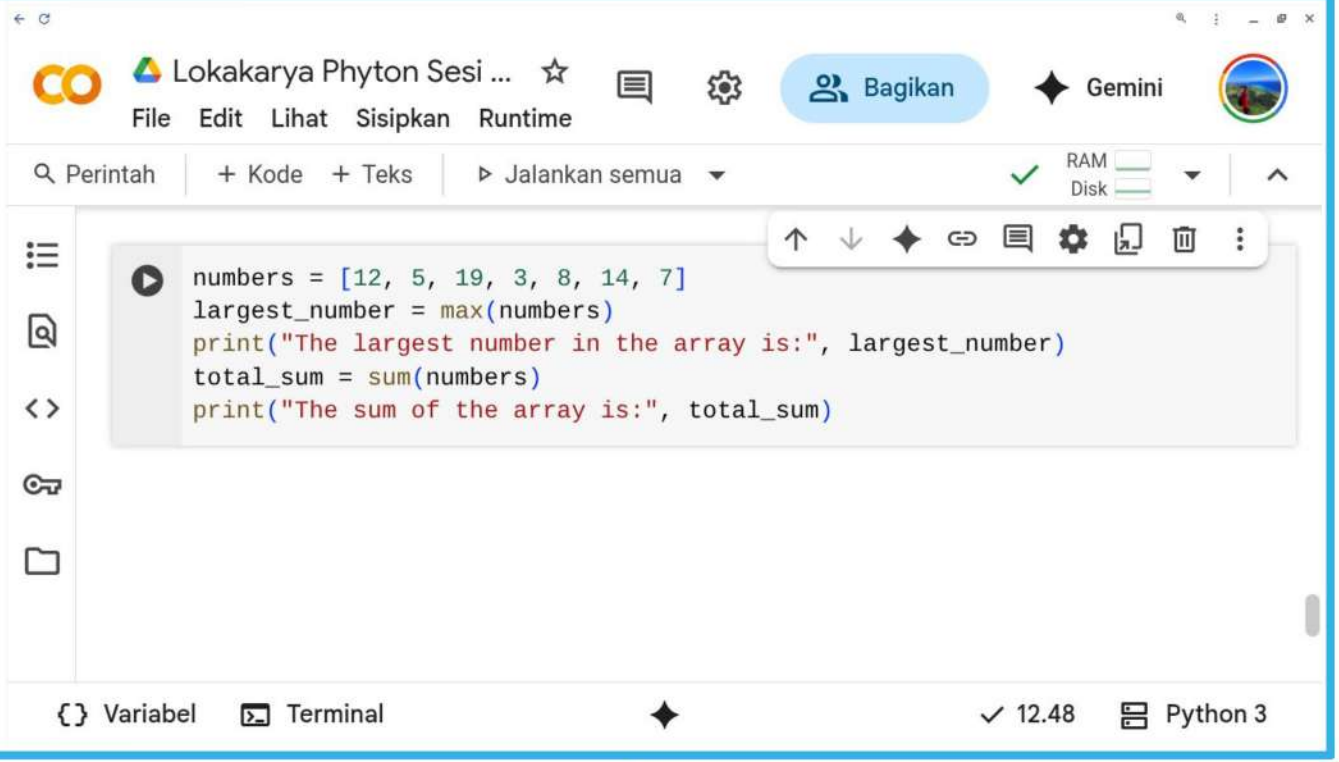
```
total_sum = sum(numbers)
```

```
print("The sum of the array is:", total_sum)
```

- **Penjelasan** : `sum()` Fungsi ini menghitung jumlah total semua elemen dalam array.

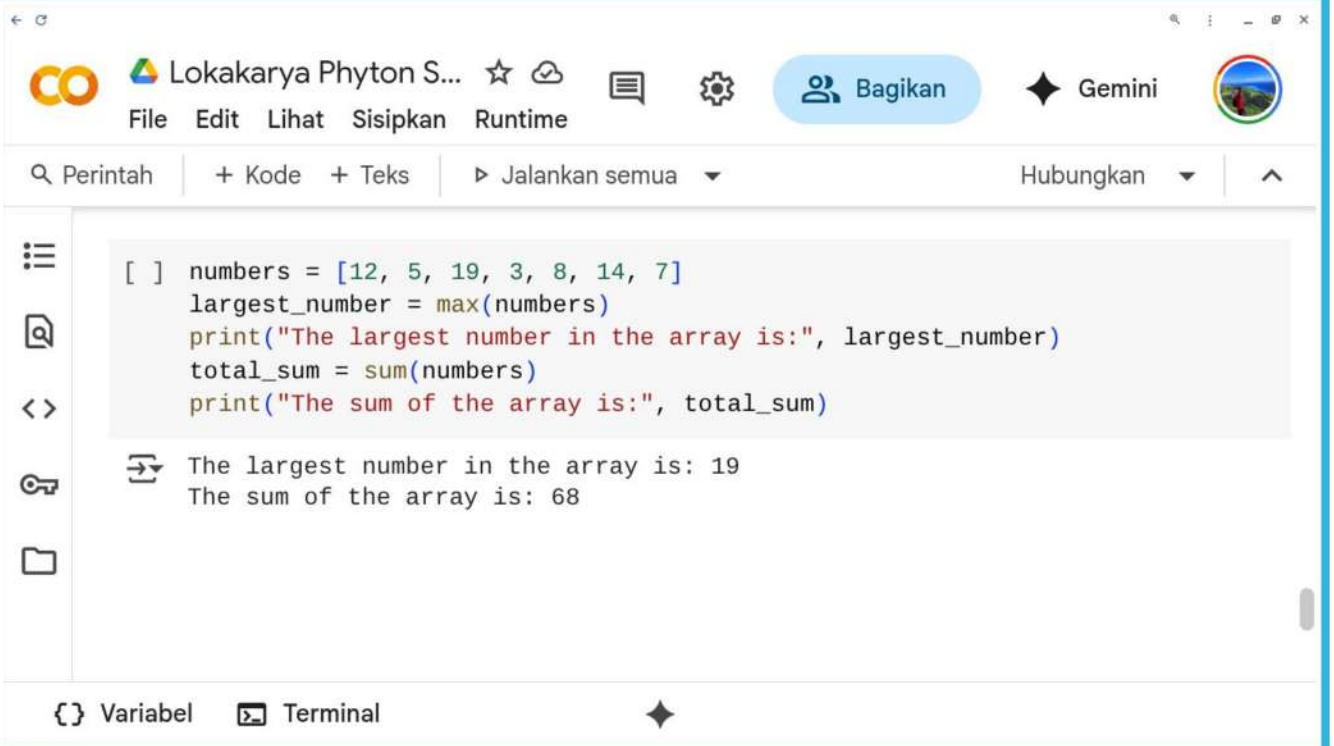
Aktivitas: Array dalam Python

Membuat Fungsi Nilai Total dan Mencetaknya



Aktivitas: Array dalam Python

Hasil Cetak Fungsi Nilai Total



Aktivitas: Array dalam Python

Langkah 4: Membalikkan Array

Anda dapat membalikkan array menggunakan metode pengirisan atau `reverse()` metode. Mari kita gunakan pengirisan untuk menyederhanakannya.

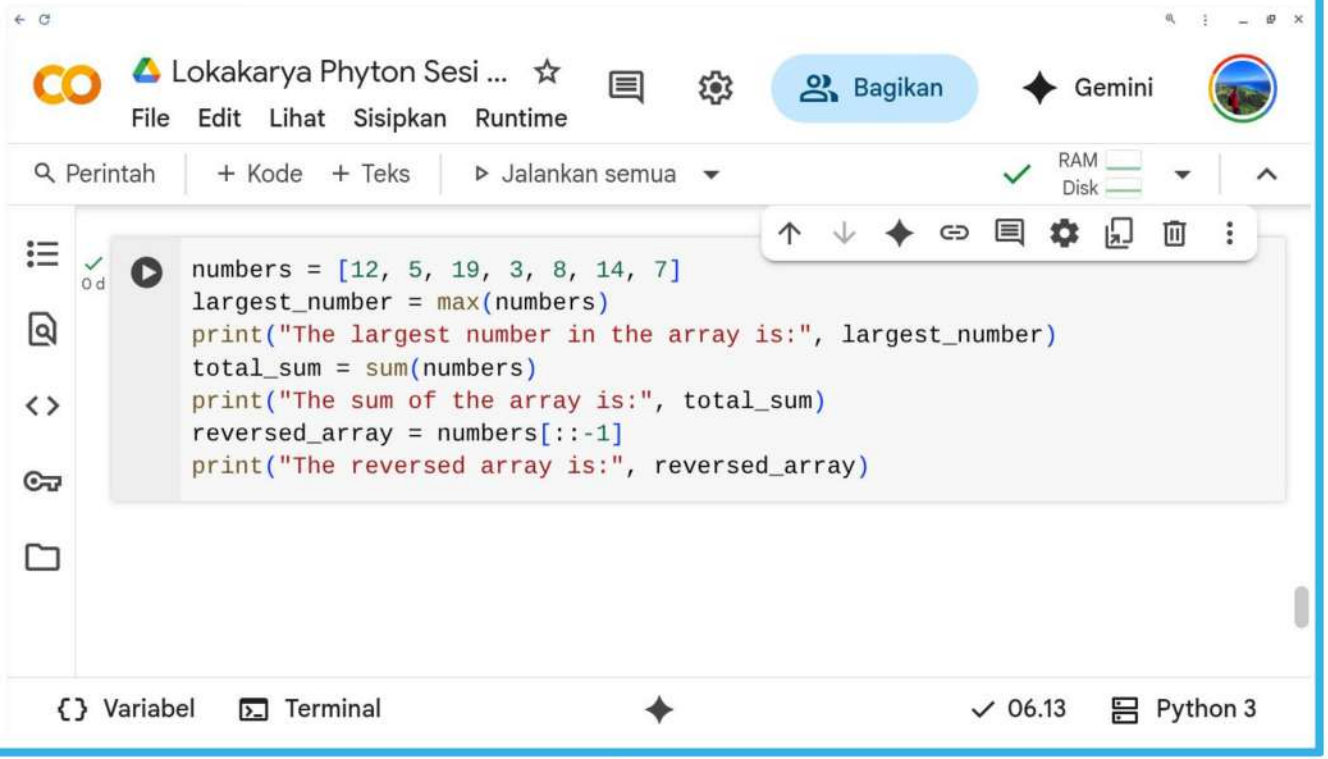
```
reversed_array = numbers[::-1]

print("The reversed array is:", reversed_array)
```

- **Penjelasan** : `[::-1]` membalikkan array dengan melangkah mundur melalui array.

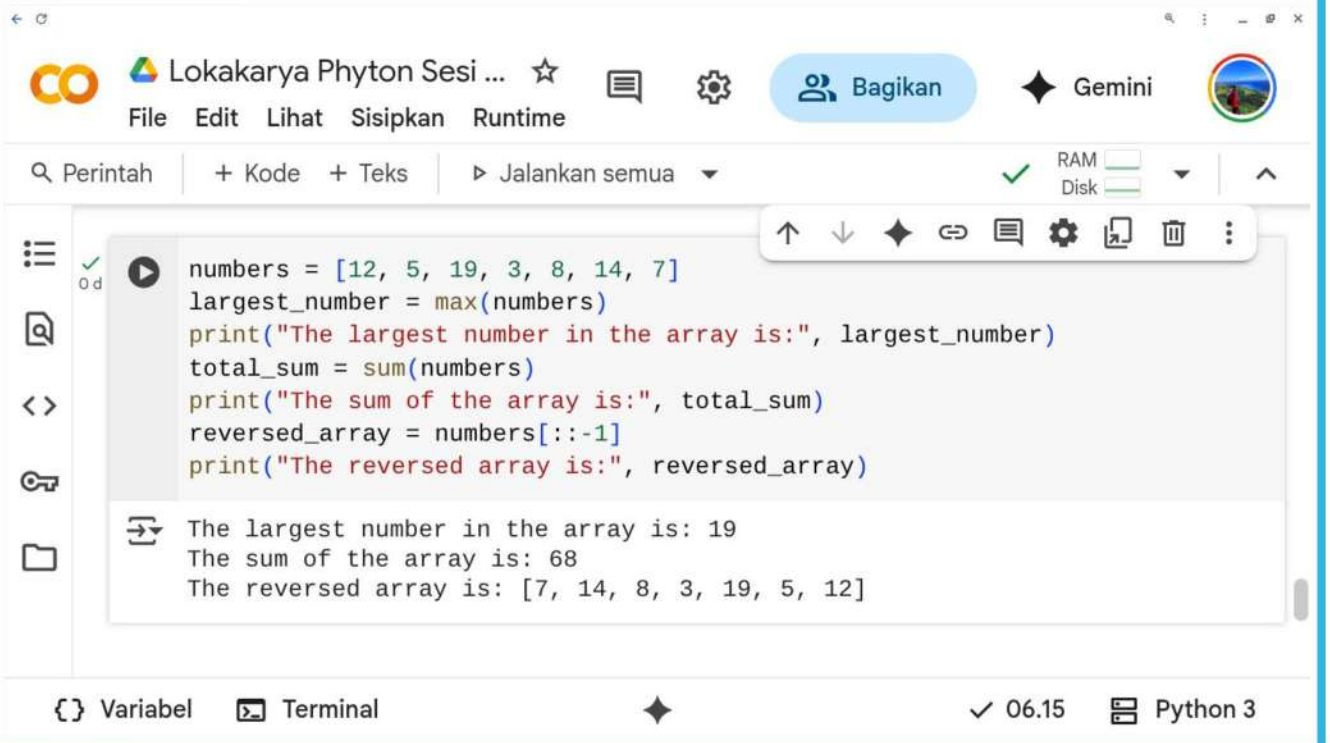
Aktivitas: Array dalam Python

Menambahkan fungsi pembalik dan mencetaknya



Aktivitas: Array dalam Python

Hasil fungsi pembalik



Aktivitas: Array dalam Python

Langkah 5: Memeriksa Apakah Nomor Tertentu Ada di Array

Kita dapat menggunakan `in` kata kunci untuk memeriksa apakah suatu angka ada dalam array.

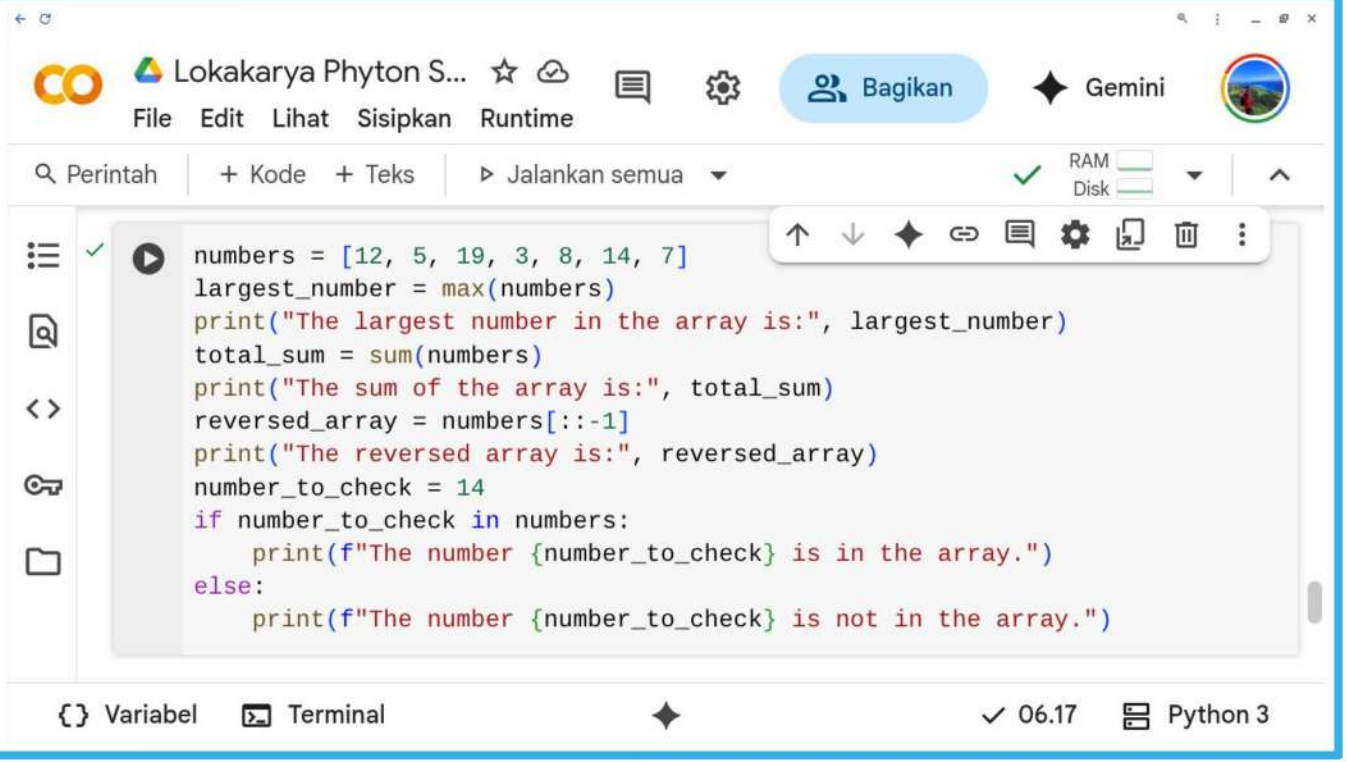
```
number_to_check = 14

if number_to_check in numbers:
    print(f"The number {number_to_check} is in the array." )
else:
    print(f"The number {number_to_check} is not in the array." )
```

- **Penjelasan** : `in` Kata kunci memeriksa apakah angka tersebut `number_to_check` ada di dalam `numbers` array. Kata kunci ini akan mengembalikan nilai `True` jika angka tersebut ditemukan, jika tidak `False`, .

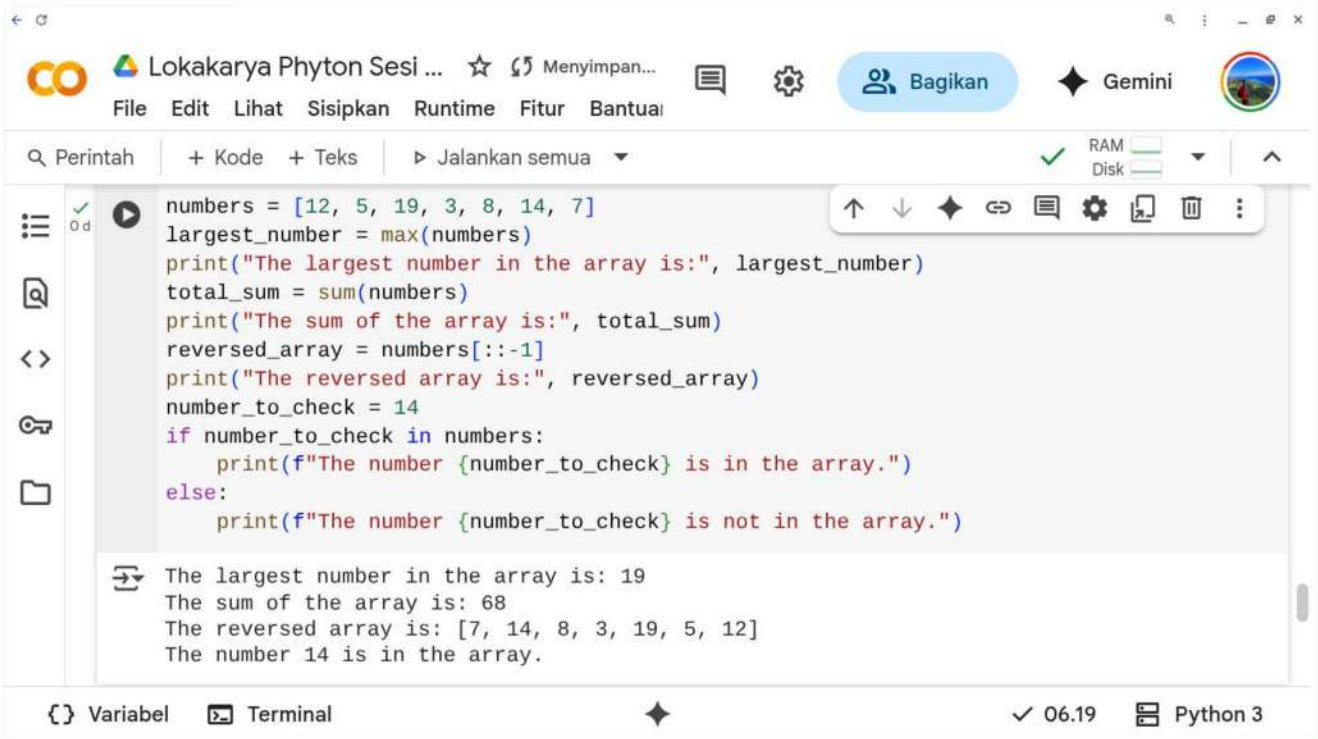
Aktivitas: Array dalam Python

Menambahkan Syarat Kondisi Pengecekan Angka Dalam Array



Aktivitas: Array dalam Python

Hasil Penerapan Kondisi Pengecekan Angka Dalam Array



Penjelasan Kode

1. **Pembuatan Array** : Kita membuat sebuah daftar `numbers yang` menampung array dari berbagai nilai.
2. **Menemukan Angka Terbesar** : Kami menggunakan fungsi bawaan `max()` untuk menemukan dan mencetak angka terbesar dalam array.
3. **Menghitung Jumlah** : Kami menggunakan fungsi bawaan `sum()` untuk menemukan jumlah array dan mencetak hasilnya.
4. **Membalikkan Array** : Kami membalikkan array menggunakan slicing (`[::-1]`) dan mencetak versi terbalik.
5. **Memeriksa Keberadaan** : Kami memeriksa apakah angka tertentu (dalam kasus ini, 14) ada dalam array menggunakan `in` operator dan mencetak pesan yang sesuai.

Tantangan Phyton

1. **Pembalikan Array**
   Buatlah program yang mengambil array angka sebagai input dan mengembalikan array dalam urutan terbalik.
2. **Menemukan Angka Maksimum dalam Suatu Larik**
   Tulislah suatu fungsi yang mencari dan mengembalikan angka terbesar dalam larik tertentu.
3. **Jumlah Array**
   Tulis fungsi yang menghitung dan mengembalikan jumlah elemen dalam array.
4. **Pencarian dalam Array**
   Buat fungsi yang memeriksa apakah angka tertentu ada dalam array dan mengembalikan `True` atau `False`.
5. **Rata-rata Elemen Array**
   Tulis fungsi yang menghitung dan mencetak rata-rata angka dalam array.